

Online Scheduling of Battery-Aware Speculative Decoding for Energy-Efficient Cloud-Edge Collaborative LLM Inference

Hengdi Wang
School of Artificial Intelligence
Beijing University of Posts and Telecommunications
Beijing, China
wanghengdi@bupt.edu.cn

Konglin Zhu*
School of Artificial Intelligence
Beijing University of Posts and Telecommunications
Beijing, China
klzhu@bupt.edu.cn

Lei Jiao*
Center for Cyber Security and Privacy
University of Oregon
Eugene, OR, USA
ljiao2@uoregon.edu

Lin Zhang
School of Artificial Intelligence
Beijing University of Posts and Telecommunications
Beijing, China
zhanglin@bupt.edu.cn

王恒迪 人工智能学院 北京邮电大学 北京，
中国 wanghengdi@bupt.edu.cn

朱孔林* 人工智能学院 北京邮电大学 北京，
中国 klzhu@bupt.edu.cn

雷娇* 俄勒冈大学电子安全与
隐私中心 美国俄勒冈州尤金 ljiao2
@uoregon.edu

张林 人工智能学院 北京邮电大学 北京，中
国 zhanglin@bupt.edu.cn

Abstract

While distributed speculative decoding can offer efficient acceleration for Large Language Model (LLM) inference in cloud-edge environments, unleashing its full potential confronts significant challenges, including complex token draft-length management, uncertain prompt arrivals and system conditions, and joint edge battery orchestration under fluctuating grid-energy prices. To systematically address these challenges, we conduct an intensive mathematical and algorithmic study. We first formulate a long-term cost minimization problem that captures all the challenges. Then, we decompose this problem into two subproblems and solve them alternately as time goes in an online manner. Particularly, we design a learning-centric algorithmic framework which employs bandit learning to address the intractability and manage draft length selection in the first subproblem and applies online learning to address the online uncertainty and manage battery states and grid-energy use in the second subproblem, both without relying on future inputs. We further conduct rigorous theoretical analysis, proving sub-linear regrets against offline optimal objectives and asymptotically-diminishing violation of long-term constraints. Extensive evaluations with real-world LLM inference demonstrate that our approach substantially outperforms several state-of-the-art methods, with many additional advantages.

Keywords

Cloud-Edge Computing, Large Language Model, Speculative Decoding, Energy Efficiency, Bandit Learning, Online Learning.

ACM Reference Format:

Hengdi Wang, Lei Jiao, Konglin Zhu, and Lin Zhang. 2026. Online Scheduling of Battery-Aware Speculative Decoding for Energy-Efficient Cloud-Edge Collaborative LLM Inference. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP '26)*, September 28-October 01, 2026, Singapore.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICPP '26, Singapore
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2657-6/2026/09
https://doi.org/10.1145/3832810.3832825

Singapore. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3832810.3832825

1 Introduction

To provision high-quality low-latency Large Language Model (LLM) inference services, the service provider leveraging cloud and edge infrastructures faces a critical trade-off: edge-only deployment is constrained by limited computational resources, while cloud-only processing introduces substantial wide-area network delay [1, 2]. Cloud-edge *speculative decoding* [3] resolves this by strategically distributing the inference pipeline, as visualized in Figure 1. That is, a lightweight *draft LLM* at the network edge auto-regressively generates an initial sequence of tokens locally, delivering rapid output to the end user; this draft is then sent to the remote cloud, where a powerful *target LLM* verifies all tokens in a single parallel forward pass, accepting correct ones and correcting the first mismatch. This architecture preserves the quality of cloud-scale models while significantly reducing the time-to-first-token and amortizing the latency over multiple tokens per communication round [4].

However, efficiently managing this distributed pipeline across the cloud-edge continuum requires the joint optimization of communication, computation, and energy usage, which is a non-trivial task due to the unique and fundamental challenges as follows.

First, the LLM inference service must dynamically manage the draft length, which directly trades computation against communication in the speculative-decoding process. The draft length impacts the number of speculative-decoding iterations between each edge and the cloud, while in every single iteration, sending too many draft tokens strains the network link and wastes computation at the edge but sending too few decreases parallelism utilization in the cloud [5, 6]. Further, the draft length at the edge needs to adapt to the target LLM’s token-acceptance rate in the cloud, requiring dynamically balancing the exploration of new different length options with the exploitation of previously-chosen lengths [7, 8]. These are all exacerbated by unpredictable inference-request arrivals, fluctuating network conditions, and pre-determined communication traffic budget, demanding smart learning-based coordination that can operate effectively without any prior knowledge.

Second, the LLM inference service must also dynamically manage the energy of the joint cloud-edge system, which constitutes a major

能量高效云边协同大语言模型推理的电池感知推测解码在线调度

王恒迪 人工智能学院 北京邮电大学 北京，
中国 wanghengdi@bupt.edu.cn

朱孔林* 人工智能学院 北京邮电大学 北京，
中国 klzhu@bupt.edu.cn

雷娇* 俄勒冈大学电子安全与
隐私中心 美国俄勒冈州尤金 ljiao2
@uoregon.edu

张林 人工智能学院 北京邮电大学 北京，中
国 zhanglin@bupt.edu.cn

摘要

虽然分布式投机解码可以在云 - 边缘环境中为大型语言模型

（LLM）推理提供高效加速，但要充分释放其潜力仍面临重大挑战，包括复杂的令牌草稿长度管理、不确定的提示到达和系统条件，以及在电网能源价格波动下对边缘电池的联合调度。为了系统地解决这些挑战，我们进行了深入的数学和算法研究。我们首先提出了一个长期成本最小化问题，以涵盖所有这些挑战。然后，我们将该问题分解为两个子问题，并随着时间的推移在线交替求解。特别地，我们设计了一个以学习为中心的算法框架，在第一个子问题中使用匪徒学习来解决不可处理性问题并管理草稿长度选择，在第二个子问题中应用在线学习以应对在线不确定性并管理电池状态和电网能源使用。

关键词

云边计算、大型语言模型、推测解码、能源效率、强盗学习、在线学习。

ACM 参考格式:

王恒迪，焦磊，朱孔林，张林 . 2026. 能量高效云 - 边缘协同大型语言模型推理的电池感知推测解码在线调度 . 载于第 55 届国际并行处理会议（ICPP '26）论文集，2026 年 9 月 28 日至 10 月 1 日

*通信作者。



本作品采用知识共享署名 4.0 国际许可协议进行许可 ICPP '26, 新加坡 © 2026 版权所有所有者 / 作者所有。ACM ISBN 979-8-4007-2657-6/2026/09
https://doi.org/10.1145/3832810.3832825

新加坡。ACM，美国纽约，11 页。https://doi.org/10.1145/3832810.3832825

1 引言

为了提供高质量、低延迟的大型语言模型（LLM）推理服务，利用云和边缘基础设施的服务提供商面临一个关键权衡：仅在边缘部署受限于有限的计算资源，而仅在云端处理会引入大量广域网延迟 [1, 2]。云 - 边缘推测解码 [3] 通过战略性地分配推理管道解决了这一问题，如图 1 所示。也就是说，位于网络边缘的轻量级草稿 LLM 以自回归方式在本地生成初始的令牌序列，为最终用户快速提供输出；然后将该草稿发送到远程云端，由强大的目标 LLM 在一次并行的前向传播中验证所有令牌，接受正确的令牌并纠正第一个不匹配。该架构在保持云端规模模型质量的同时，显著减少首次令牌生成时间，并将延迟在多个令牌中进行摊销。

然而，高效管理跨云边缘连续体的分布式流水线需要对通信、计算和能量使用进行联合优化，这是一项非平凡的任务，因为存在以下独特且根本性的挑战。

首先，LLM 推理服务必须动态管理草稿长度，这直接在推测解码过程中权衡计算量与通信量。草稿长度会影响每次边缘与云之间的推测解码迭代次数，而在每次迭代中，发送过多的草稿 token 会加重网络负担并浪费边缘计算资源，但发送过少又会降低云端的并行利用率 [5, 6]。此外，边缘的草稿长度需要适应目标 LLM 在云端的 token 接收速率，这需要在探索新的不同长度选项与利用之前选择的长度之间进行动态平衡 [7, 8]。这些问题都因不可预测的推理请求到达、波动的网络状况以及预定的通信流量预算而被加剧，这就要求基于智能学习的协调机制能够在没有任何先验知识的情况下有效运行。

其次，LLM 推理服务还必须动态管理联合云 - 边缘系统的能量，这构成了一个主要的

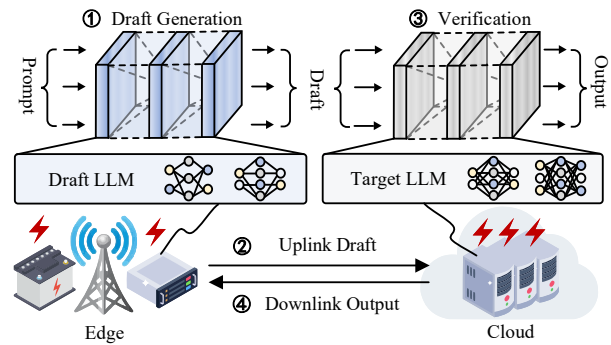


Figure 1: System scenario

source of the operational expenditure nowadays. In particular, this is not just only about how the complexity of speculative decoding influences the grid-energy consumption for the cloud, but also about how it could entail the charging and discharging operations for the batteries widely equipped at today's edges (e.g., 5G edge servers) [9, 10]. Edge battery-energy storage has the advantage of shifting energy footprint to low-price periods, but has limited capacity and degradation with use [11]. The core dilemma is whether to conserve energy from the grid now even at a high price, or to use the stored energy now which risks battery depletion and even higher grid-energy cost in the future, while jointly managing the speculative-decoding-based LLM inference. Decisions must be made online without future energy price and request loads.

Unfortunately, existing research has inadequately addressed the aforementioned challenges. Studies [1, 4, 12–14] that provide LLM inference services through cloud-edge collaboration often overlook the energy consumption of LLMs and the potential of speculative decoding to accelerate inference. Research [5, 6, 15–17] on LLM speculative decoding in resource-constrained networks either focuses solely on communication without considering the impact of energy consumption, or never accounts for limited communication resources. Those [9, 10, 18–20] investigating edge battery management also neglect the cloud-edge LLM inference scenario. Detailed discussions of related work are in Section 6.

This paper, for the first time, presents a rigorous modeling and mathematical study on the joint online scheduling of speculative decoding and battery energy for the cloud-edge collaborative LLM inference systems. We make four-fold contributions:

First, we model and formulate a long-term optimization problem that minimizes the total cost of cloud-edge LLM inference. Our models capture the end-to-end distributed speculative decoding process, including the computation delay of edge generation and cloud verification, the communication delay of token transference between edge and cloud, and the cloud-edge energy expense with edge battery degradation. Our problem turns out to be a nonlinear mixed-integer program, pursuing optimization over diverse inference requests while respecting the communication budget with sufficient energy provisioning. Our formulation is general and readily accommodates the dynamism and heterogeneity of inputs such as inference requests, energy prices, and battery degradation costs.

Second, we devise a novel learning-centric decomposition-based algorithmic framework that operates in polynomial time to solve this problem online. Our approach decomposes the original problem

into two subproblems, containing draft length control and energy control, respectively. For the former subproblem, we design a *bandit learning* [21, 22] approach that, for each edge, constructs a feasible set of draft length options under the communication budget and updates the weights of the selected draft lengths through exponential weighting as time goes, using such weights to dynamically manage the trade-off between exploring new draft lengths and exploiting existing draft lengths. For the latter subproblem, with the given draft length, we design an *online learning* [23, 24] approach to manage battery operations and grid-energy use, which utilizes the convex-concave reformulation and the alternate primal-dual updates facilitated by a virtual queue, without relying on future inputs. Overall, our key algorithmic strategy is to group and isolate the different challenges, namely *integrality*, *posterior*, *state transitions*, and *online uncertainty*, in separate subproblems, and identify and leverage different algorithmic techniques to address them.

Third, we derive rigorous theoretical analysis for our algorithms. For the first subproblem, we prove that the *regret*, i.e., the gap between the expected inference delay incurred by our online draft length selections and that incurred by the single best draft length grows slower than the natural progression of time. For the second subproblem, we also prove that, for the joint energy expense and battery degradation, the *regret*, i.e., the difference between the objective value from our online energy decisions and that from the series of the per-slot optimal energy decisions, possesses sub-linear growth with regard to the length of the time horizon, and that the *violation* of the long-term constraint due to battery state transitions is sub-linear, i.e., diminishing asymptotically. Finally, combining these results through introducing intermediate auxiliary terms, we prove that our original problem can adopt a sub-linear upper bound upon its *regret* with regard to overall static offline optimal decisions.

Fourth, we conduct extensive experiments using real LLM inference datasets [25–29], draft-target LLM pairs [7, 8], workload traces [30], grid-energy prices [31], and edge locations [32], as well as cloud-edge communication conditions [1, 33], capacity [11], efficiency [9], and battery degradation parameters [34]. We compare our proposed approach against multiple state-of-the-art alternatives under various settings, and obtain the following results: (i) our approach reduces the total cost by about 20%~50% compared to others; (ii) the regrets and the violation grow sub-linearly in practice, aligned with our theoretical analysis; (iii) our approach scales with robustness as workload and communication budget vary; (iv) by adjusting weights and other parameters, our approach can successfully navigate optimization trade-offs between inference latency and energy consumption; (v) our algorithms finish execution within tens of milliseconds in each one-minute time slot, fully acceptable in reality; (vi) as grid-energy price fluctuates, the battery energy is well utilized by our algorithms to jointly supply adequate energy to LLM inference at minimum total expense.

2 Modeling and Formulation

2.1 System Settings and Models

We summarize our main notations and descriptions in Table 1.

Cloud-Edge LLM Infrastructure: We target an LLM service provider that owns a cloud-edge infrastructure and uses this infrastructure to offer the LLM inference service to the end users. This

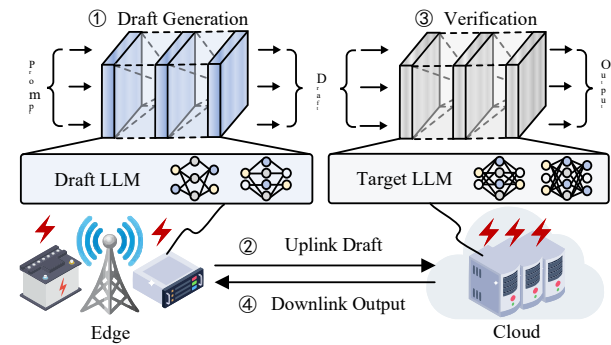


图 1: 系统场景

如今运营支出的来源。特别是，这不仅仅涉及投机解码的复杂性如何影响云端的电网能耗，还涉及其可能导致目前广泛配备在边缘的电池（例如 5G 边缘服务器）的充放电操作。边缘电池储能的优势在于可以将能耗转移到低价时段，但其容量有限且使用过程中会退化。核心的困境是，是现在以高价从电网节约能源，还是使用存储的能源，这可能导致电池耗尽，甚至未来电网能源成本更高，同时需要共同管理基于投机解码的 LLM 推理。决策必须在线进行，无法提前得知未来的能源价格和请求负载。

不幸的是，现有的研究对前述挑战的解决不足。通过云边协作提供大语言模型（LLM）推理服务的研究通常忽略了 LLM 的能耗以及投机解码加速推理的潜力。在资源受限网络中关于 LLM 投机解码的研究，要么仅关注通信而不考虑能耗的影响，要么根本不考虑有限的通信资源。那些研究边缘设备电池管理的工作也忽略了云边 LLM 推理场景。相关工作的详细讨论见第 6 节。

本文首次对云边协作大语言模型推理系统中推测解码与电池能量的联合在线调度进行了严格的建模和数学研究。我们做出了四方面的贡献：

首先，我们建立并 formulat 一个长期优化问题，旨在最小化云 - 边缘 LLM 推理的总成本。我们的模型捕捉了端到端的分布式预测解码过程，包括边缘生成和云验证的计算延迟、边缘与云之间令牌传输的通信延迟，以及伴随边缘电池衰减的云 - 边能源消耗。我们的优化问题最终成为一个非线性混合整数规划问题，旨在对多样的推理请求进行优化，同时在具备足够能源供应的前提下遵守通信预算。我们的公式具有通用性，并可轻松适应输入的动态性和异质性，例如推理请求、能源价格和电池衰减成本。

其次，我们设计了一种基于学习的全新分解算法框架，该框架能够在多项式时间内在线解决此问题。我们的方法对原始问题进行分解

分为两个子问题，分别涉及草稿长度控制和能量控制。对于前一个子问题，我们设计了一种强盗学习 [21, 22] 方法，对于每条边，在通信预算下构建可行的草稿长度选项集合，并随着时间的推移通过指数加权更新所选草稿长度的权重，利用这些权重动态管理探索新草稿长度与利用现有草稿长度之间的权衡。对于后一个子问题，在给定草稿长度的情况下，我们设计了一种在线学习 [23, 24] 方法来管理电池操作和电网能源使用，该方法利用凸 - 凹重构和由虚拟队列促成的交替原始 - 对偶更新，而不依赖于未来输入。总体而言，我们的关键算法策略是对不同挑战进行分组和隔离，即整数性、后验、状态转移和在线不确定性

第三，我们对我们的算法进行了严格的理论分析。对于第一个子问题，我们证明了遗憾，即由我们在线草稿长度选择引起的预期推理延迟与单个最佳草稿长度引起的延迟之间的差距，其增长速度慢于时间的自然进程。对于第二个子问题，我们也证明了，对于联合能耗和电池衰退，遗憾，即我们的在线能量决策的目标值与每时隙最优能量决策序列的目标值之间的差异，随着时间范围的长度呈次线性增长，并且由于电池状态转换引起的长期约束的违规也是次线性的，即渐近地减小。最后，通过引入中间辅助项将这些结果结合起来，我们证明了我们的原始问题可以对其遗憾采用次线性上界。

第四，我们使用真实的 LLM 推理数据集 [25、29]，草稿 - 目标 LLM 对 [7, 8]，工作负载轨迹 [30]，电网能源价格 [31]，和边缘位置 [32]，以及云 - 边缘通信条件 [1, 33]，容量 [11]，效率 [9]，和电池退化参数 [34] 进行了广泛的实验。我们在各种设置下将我们提出的方法与多种最先进的替代方案进行比较，并获得以下结果：(i) 与其他方法相比，我们的方法将总成本降低了约 20%~50；(ii) 实际上，遗憾值和违规情况呈次线性增长，与我们的理论分析一致；(iii) 当工作负载和通信预算变化时，我们的方法具有可扩展的鲁棒性；(iv) 通过调整权重和其他参数，我们的方法可以成功在推理延迟和能耗之间导航优化权衡；(v) 我们的算法在每个一分钟内的执行时间都在几十毫秒内完成。

2 建模与公式化

2.1 系统设置与模型

我们在表 1 中总结了我们的主要符号和描述。

云 - 边缘大语言模型基础设施: 我们面向拥有云 - 边缘基础设施的 LLM 服务提供商，并利用该基础设施向最终用户提供 LLM 推理服务。

Table 1: Notations and Descriptions

Input	Description
$\mathcal{I}$	Set of inference requests
$\mathcal{N}$	Set of edges
$\mathcal{T}$	Set of time slots
$\mathbf{h}_{i,n}$	Context for request i on edge n
$\mathbf{u}^{\text{draft}}$	Sequence of draft tokens on edge
$\mathbf{q}^{\text{draft}}$	Sequence of probabilities for draft tokens
$\mathbf{u}^{\text{target}}$	Output sequence in cloud
u_k	k -th draft token in draft sequence $\mathbf{u}^{\text{draft}}$
κ	First position where rejection occurs in acceptance test
$\tilde{u}_\kappa$	Corrected or newly-generated token at position κ
v_k	Random value sampled for acceptance at position k
p_k^{draft}	Probability of k -th draft token by draft LLM (abbreviated as q_k)
p_k^{target}	Probability of k -th draft token by target LLM (abbreviated as p_k)
f_n^{draft}	Conditional probability distribution function of draft LLM on edge n
f_n^{target}	Conditional probability distribution function of target LLM in cloud
α	Probability of accepting a draft token by target LLM
U_{nt}	Number of accepted draft tokens per iteration on edge n at t
G_{nt}	Number of output tokens per iteration on edge n at t
M_{int}	Total length of output sequence for request i on edge n at t
T_n'	Time consumed for auto-regressively generating a token on edge n
D_n	Communication set-up time between edge n and cloud
β_n	Time consumed for edge n to transfer a token with probability to cloud
β_n'	Time per token for cloud to transfer output sequence to edge n
β_n''	Network traffic of sending a draft token and probability from edge n
T_n''	Time consumed by a parallel verification for draft tokens in cloud
φ_{int}	Total delay for resolving request i on edge n at t
T_{nt}	Total delay of all steps per iteration on edge n at t
ρ_{int}	Network traffic incurred between edge n and cloud for request i at t
A_n	Budget on total cumulative traffic of edge n
ξ_{jnt}	Energy of edge n for generating draft tokens for request i at t
ξ_{int}	Energy of edge n for transmitting draft tokens for request i at t
ψ_{int}	Energy of cloud for handling draft tokens of request i from edge n at t
P_n, Q_n	Energy efficiency constants depending on draft LLM on edge n
P, Q	Energy efficiency constants depending on target LLM in cloud
r_n	Energy of edge n per single-token transmission
s_t	Energy price of power grid at t
a_n	Minimum battery energy limit of edge n
b_{nt}	Available battery energy of edge n at t
c_n	Maximum battery energy limit of edge n
m_n	Acquisition price of battery for edge n
o_n	Maximum number of cycles that battery of edge n can operate
d_n	Degradation cost per unit energy charged or discharged for edge n
η_c	Battery charging efficiency
η_d	Battery discharging efficiency
Decision	Description
Y_{nt}	Number of draft tokens to generate per iteration on edge n at t
x_{nt}	Amount of grid energy to charge battery of edge n at t
y_{nt}	Amount of energy to discharge from battery of edge n at t
z_{nt}	Amount of grid energy for supporting LLM inference on edge n at t
e_t	Amount of grid energy for supporting LLM inference in cloud at t

infrastructure consists of a cloud data center and a group of distributed and potentially heterogeneous “edges”, where each edge is a micro data center or server cluster co-located with a cellular base station or WiFi access point in close proximity to the end users. The cloud and the edges are interconnected via wired networks, and the end users connect to the edges via wireless or wired networks. We denote the set of edges as $\mathcal{N} = \{1, 2, \dots, N\}$. We consider the system operating over a series of time slots $\mathcal{T} = \{1, 2, \dots, T\}$.

LLM Inference via Speculative Decoding: Without loss of generality, the distributed speculative-decoding-based LLM inference service has a full-fledged powerful *target LLM* run in the cloud and one lightweight and often smaller *draft LLM* run on each edge.

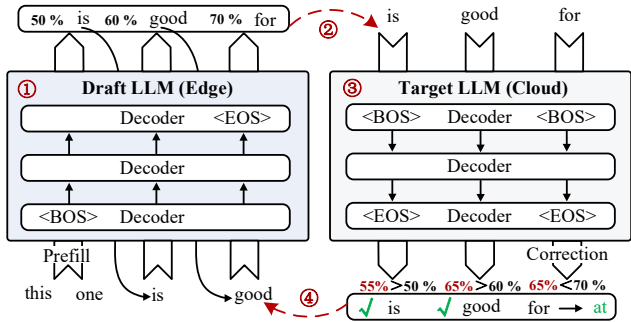


Figure 2: Indicative example of speculative decoding: The draft LLM autoregressively generates draft tokens “is good for” (Step 1), and sends them to the target LLM (Step 2); The target LLM verifies and corrects them (Step 3), and returns output tokens “is good at” to the draft LLM (Step 4).

At each time slot t , for each inference request $i \in \mathcal{I} = \{1, 2, \dots, I\}$ (i.e., a sequence of input tokens) that arrives at the edge n from the end user, the edge n sends the context $\mathbf{h}_{i,n}$ (e.g., input tokens from the user, plus other system prompts if needed) to the cloud and afterwards the LLM inference (i.e., generating the sequence of output tokens) using *speculative decoding* is an iterative process between the edge n and the cloud, where each iteration is composed of multiple steps described as follows (also see Figure 2):

- **Step 1:** The draft LLM on the edge n auto-regressively generates a speculative draft token sequence of the length $\gamma_{nt} \in \{1, \dots, \gamma_{\max}\}$ as pre-specified, denoted as $\mathbf{u}^{\text{draft}} \triangleq \{u_1, \dots, u_{\gamma_{nt}}\}$, where $u_k, \forall k \in \{1, \dots, \gamma_{nt}\}$ refers to the k -th draft token. Each $u_k, \forall k$ is sampled from the conditional probability distribution $p_k^{\text{draft}}(\cdot) \triangleq f_n^{\text{draft}}(\cdot | \mathbf{h}_{i,n}, \mathbf{u}_{1:k-1}^{\text{draft}})$, where f_n^{draft} depends on the draft LLM; $\mathbf{h}_{i,n}$ is the context so far; and $\mathbf{u}_{1:k-1}^{\text{draft}}$ is the sequence of draft tokens before the k -th draft token. The edge n also extracts the actual probability of the draft token $u_k, \forall k$ as $q_k \triangleq p_k^{\text{draft}}(u_k)$, thereby producing $\mathbf{q}^{\text{draft}} \triangleq \{q_1, \dots, q_{\gamma_{nt}}\}$.
- **Step 2:** The edge n sends $\mathbf{u}^{\text{draft}}$ and $\mathbf{q}^{\text{draft}}$ to the cloud.
- **Step 3:** The target LLM in the cloud processes the entire sequence $\mathbf{u}^{\text{draft}}$ in one parallel forward pass to compute the conditional probability distribution for each position k as $p_k^{\text{target}}(\cdot) \triangleq f_n^{\text{target}}(\cdot | \mathbf{h}_{i,n}, \mathbf{u}_{1:k-1}^{\text{draft}})$, where f_n^{target} depends on the target LLM. The cloud then performs the acceptance test sequentially. That is, for each position k , it calculates the probability $p_k \triangleq p_k^{\text{target}}(u_k)$, samples a random value $v_k \sim \text{Uniform}(0, 1)$, and accepts the draft token u_k if $v_k \leq \min\{1, \frac{p_k}{q_k}\}$. Let κ be the first position where rejection occurs, i.e., $\kappa \leq \gamma_{nt}$ or $\kappa = \gamma_{nt} + 1$. The output sequence is $\mathbf{u}^{\text{target}} \triangleq \{u_1, \dots, u_{\kappa-1}, \tilde{u}_\kappa\}$, where $\tilde{u}_\kappa \sim p_\kappa^{\text{target}}(\cdot)$ is the corrected or newly-generated token. The cloud updates the context as $\mathbf{h}_{i,n} \leftarrow \mathbf{h}_{i,n} \oplus \mathbf{u}^{\text{target}}$.
- **Step 4:** The cloud sends the output sequence $\mathbf{u}^{\text{target}}$ to the edge n , and the edge n updates the context as $\mathbf{h}_{i,n} \leftarrow \mathbf{h}_{i,n} \oplus \mathbf{u}^{\text{target}}$, preparing for the next iteration.

Without loss of generality and following existing research [3], we consider the expected number of tokens in the output sequence produced per iteration in speculative decoding for each inference

表 1: 符号与说明

Input	Description
$\mathcal{I}$	Set of inference requests
$\mathcal{N}$	Set of edges
$\mathcal{T}$	Set of time slots
$\mathbf{h}_{i,n}$	Context for request i on edge n
$\mathbf{u}^{\text{draft}}$	Sequence of draft tokens on edge
$\mathbf{q}^{\text{draft}}$	Sequence of probabilities for draft tokens
$\mathbf{u}^{\text{target}}$	Output sequence in cloud
u_k	k -th draft token in draft sequence $\mathbf{u}^{\text{draft}}$
κ	First position where rejection occurs in acceptance test
$\tilde{u}_\kappa$	Corrected or newly-generated token at position κ
v_k	Random value sampled for acceptance at position k
p_k^{draft}	Probability of k -th draft token by draft LLM (abbreviated as q_k)
p_k^{target}	Probability of k -th draft token by target LLM (abbreviated as p_k)
f_n^{draft}	Conditional probability distribution function of draft LLM on edge n
f_n^{target}	Conditional probability distribution function of target LLM in cloud
α	Probability of accepting a draft token by target LLM
U_{nt}	Number of accepted draft tokens per iteration on edge n at t
G_{nt}	Number of output tokens per iteration on edge n at t
M_{int}	Total length of output sequence for request i on edge n at t
T_n'	Time consumed for auto-regressively generating a token on edge n
D_n	Communication set-up time between edge n and cloud
β_n	Time consumed for edge n to transfer a token with probability to cloud
β_n'	Time per token for cloud to transfer output sequence to edge n
β_n''	Network tra c of sending a draft token and probability from edge n
T_n''	Time consumed by a parallel veri cation for draft tokens in cloud
φ_{int}	Total delay for resolving request i on edge n at t
T_{nt}	Total delay of all steps per iteration on edge n at t
ρ_{int}	Network tra c incurred between edge n and cloud for request i at t
A_n	Budget on total cumulative tra c of edge n
ξ_{jnt}	Energy of edge n for generating draft tokens for request i at t
ξ_{int}	Energy of edge n for transmitting draft tokens for request i at t
ψ_{int}	Energy of cloud for handling draft tokens of request i from edge n at t
P_n, Q_n	Energy e ciency constants depending on draft LLM on edge n
P, Q	Energy e ciency constants depending on target LLM in cloud
r_n	Energy of edge n per single-token transmission
s_t	Energy price of power grid at t
a_n	Minimum battery energy limit of edge n
b_{nt}	Available battery energy of edge n at t
c_n	Maximum battery energy limit of edge n
m_n	Acquisition price of battery for edge n
o_n	Maximum number of cycles that battery of edge n can operate
d_n	Degradation cost per unit energy charged or discharged for edge n
η_c	Battery charging e ciency
η_d	Battery discharging e ciency
Decision	Description
Y_{nt}	Number of draft tokens to generate per iteration on edge n at t
x_{nt}	Amount of grid energy to charge battery of edge n at t
y_{nt}	Amount of energy to discharge from battery of edge n at t
z_{nt}	Amount of grid energy for supporting LLM inference on edge n at t
e_t	Amount of grid energy for supporting LLM inference in cloud at t

基础设施由一个云数据中心和一组分布式且可能异构的 “边缘” 组成，其中每个边缘是与蜂窝基站或 WiFi 接入点共址的微型数据中心或服务器集群，靠近最终用户。云和边缘通过有线网络互连，而最终用户通过无线或有线网络连接到边缘。我们将边缘集合表示为 $\mathcal{N} = \{1, 2, \dots, N\}$ 。我们考虑系统在一系列时间槽 $\mathcal{T} = \{1, 2, \dots, T\}$ 上运行。

通过推测解码进行 LLM 推理: 在不丧失一般性的前提下，基于分布式推测解码的 LLM 推理服务在云端运行一个功能完备强大的目标 LLM，并在每个边缘设备上运行一个轻量且通常较小的草稿 LLM。

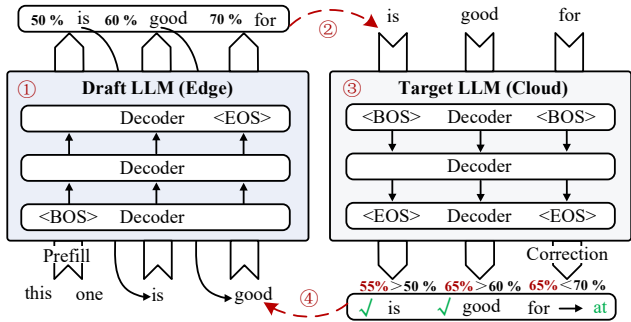


图 2: 投机解码的指示性示例: draftLLM 自回归生成草稿标记 is good for （步骤 1），并将其发送到目标 LLM （步骤 2）; 目标 LLM 验证并纠正它们（步骤 3），并将输出标记 is good at 返回给 draft LLM （步骤 4）。

在每一个时间片 t ，对于每个到达边缘 n 的推理请求 $i \in \mathcal{I} = \{1, 2, \dots, I\}$ （即一系列输入令牌），边缘 n 将上下文 $\mathbf{h}_{i,n}$ （例如来自用户的输入令牌，以及必要时的其他系统提示）发送到云端，之后使用推测解码进行的 LLM 推理（即生成输出令牌序列）是边缘 n 与云端之间的迭代过程，其中每次迭代由以下多个步骤组成（参见图 2）：

- 步骤 1: 边缘上的草稿大语言模型 n 自回归地生成一个长度为 $\gamma_{nt} \in \{1, \dots, \gamma_{\max}\}$ 的推测草稿令牌序列，如预先指定，记作 $\mathbf{u}^{\text{draft}} \triangleq \{u_1, \dots, u_{\gamma_{nt}}\}$ ，其中 $u_k, \forall k \in \{1, \dots, \gamma_{nt}\}$ 表示第 k 个草稿令牌。每个 $u_k, \forall k$ 都从条件概率分布 $p_k^{\text{draft}}(\cdot) \triangleq f_n^{\text{draft}}(\cdot | \mathbf{h}_{i,n}, \mathbf{u}_{1:k-1}^{\text{draft}})$ 中采样，其中 f_n^{draft} 取决于草稿大语言模型； $\mathbf{h}_{i,n}$ 是迄今为止的上下文；而 $\mathbf{u}_{1:k-1}^{\text{draft}}$ 是第 k 个草稿令牌之前的草稿令牌序列。边缘还提取草稿令牌 $u_k, \forall k$ 的实际概率，记作 $q_k \triangleq p_k^{\text{draft}}(u_k)$ ，从而生成 $\mathbf{q}^{\text{draft}} \triangleq \{q_1, \dots, q_{\gamma_{nt}}\}$ 。
- 步骤 2: 边缘 n 将 $\mathbf{u}^{\text{draft}}$ 和 $\mathbf{q}^{\text{draft}}$ 发送到云端。
- 步骤 3: 云端的目标大语言模型（LLM）对整个序列 $\mathbf{u}^{\text{draft}}$ 进行一次并行前向计算，以计算每个位置 k 的条件概率分布 p_k^{target} 为 $p_k^{\text{target}}(\cdot) \triangleq f_n^{\text{target}}(\cdot | \mathbf{h}_{i,n}, \mathbf{u}_{1:k-1}^{\text{draft}})$ ，其中 f_n^{target} 取决于目标 LLM。然后，云端依次执行接受测试。也就是说，对于每个位置 k ，它计算概率 $p_k \triangleq p_k^{\text{target}}(u_k)$ ，抽取一个随机值 $v_k \sim \text{Uniform}(0, 1)$ ，并在 $v_k \leq \min\{1, \frac{p_k}{q_k}\}$ 时接受草稿标记 u_k 。设 κ 为第一次拒绝发生的位置，即 $\kappa \leq \gamma_{nt}$ 或 $\kappa = \gamma_{nt} + 1$ 。输出序列是 $\mathbf{u}^{\text{target}} \triangleq \{u_1, \dots, u_{\kappa-1}, \tilde{u}_\kappa\}$ ，其中 $\tilde{u}_\kappa \sim p_\kappa^{\text{target}}(\cdot)$ 是被纠正或新生成的标记。云端将上下文更新为 $\mathbf{h}_{i,n} \leftarrow \mathbf{h}_{i,n} \oplus \mathbf{u}^{\text{target}}$ 。
- 步骤 4: 云将输出序列 $\mathbf{u}^{\text{target}}$ 发送到边缘 n ，边缘 n 将上下文更新为 $\mathbf{h}_{i,n} \leftarrow \mathbf{h}_{i,n} \oplus \mathbf{u}^{\text{target}}$ ，为下一次迭代做准备。

在不失一般性的前提下，并遵循现有研究 [3]，我们考虑在每次推理中，投机解码生成的输出序列中期望的令牌数量

request from the edge n at the time slot t . Denote α as the probability for the cloud to accept a single draft token. Let U_{nt} be the number of accepted draft tokens, and G_{nt} be the number of tokens in the output sequence. Then, we have¹ $\mathbb{E}[G_{nt}] = 1 + \mathbb{E}[U_{nt}] = 1 + \sum_{k=1}^{Y_{nt}} \mathbb{P}[U_{nt} \geq k] = 1 + \sum_{k=1}^{Y_{nt}} \alpha^k = 1 + \alpha \cdot \frac{1-\alpha^{Y_{nt}+1}}{1-\alpha} = \frac{1-\alpha^{Y_{nt}+1}}{1-\alpha}$.

Delay of LLM Inference: We account for the total computation and communication delay for resolving the inference request i via distributed speculative decoding between the edge n and the cloud at the time slot t . Note that the delay and the traffic incurred by transferring the *context* of each inference request to the cloud naturally relies on system inputs and not on the control decisions considered in this work, also explained next.

First, given M_{int} as the total length of the output sequence, i.e., the total number of output tokens for responding to this inference request, the number of cloud-edge speculative decoding iterations for this inference request is $M_{int}/\mathbb{E}[G_{nt}]$.

Second, each iteration consists of *Steps 1~4* as stated, and for each of such iterations, we have the following: (i) Computation delay of *Step 1* is $\gamma_{nt}T'_n$, where T'_n represents the time consumed for all operations related to auto-regressively generating a single token on the edge n ; (ii) Communication delay of *Step 2* is $D_n + \beta_n\gamma_{nt}$, where D_n is the communication set-up time between the edge n and the cloud, and β_n is the time consumed for the edge n to transfer a single token with its probability to the cloud; (iii) Computation delay of *Step 3* is T'' , which is the time consumed in the cloud by a single parallel forward pass for draft token verifications (note that the sequential acceptance test is often negligible in time compared to the forward pass); (iv) Communication delay of *Step 4* is $D_n + \beta'_n\mathbb{E}[G_{nt}]$, where β'_n is the time per token for the cloud to transfer the output sequence to the edge n .

Therefore, the total delay, written as a function of γ_{nt} , is

$$\varphi_{int}(\gamma_{nt}) \triangleq M_{int}/\mathbb{E}[G_{nt}] \cdot T_{nt}, \quad (1)$$

where $T_{nt} = \gamma_{nt}T'_n + 2D_n + \beta_n\gamma_{nt} + T'' + \beta'_n\mathbb{E}[G_{nt}]$ is the total delay of all steps per iteration.

Traffic of LLM Inference: For the speculative-decoding-based LLM inference for each inference request i from the edge n at the time slot t , the network traffic incurred between the edge n and the cloud, written as a function of γ_{nt} , is

$$\rho_{int}(\gamma_{nt}) \triangleq M_{int}/\mathbb{E}[G_{nt}] \cdot (\beta'_n\gamma_{nt} + \mathbb{E}[G_{nt}]), \quad (2)$$

where β'_n refers to the amount of network traffic incurred by sending a single draft token and its probability. To prevent the consumption of excessive communication resource, we impose a budget A_n on the total cumulative traffic of edge n at every t .

Energy of LLM Inference: First, the energy consumption of the edge includes the energy of computation and the energy of transmission. For the inference request i that arrives at the edge n at the time slot t , the draft LLM performs “prefill” and “decode” operations to auto-regressively generate the draft tokens, which consumes the energy of $\xi'_{int}(\gamma_{nt}) \triangleq P_n \cdot |\mathbf{h}_{i,n}|^2 + Q_n \cdot |\mathbf{h}_{i,n}| \cdot M_{int}/\mathbb{E}[G_{nt}] \cdot \gamma_{nt}$, where P_n and Q_n are constants depending on the draft LLM, the server energy efficiency, etc.; the energy for transmitting all draft

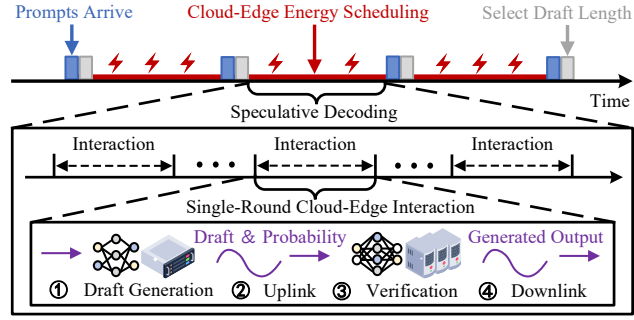


Figure 3: Time flow of cloud-edge system

tokens to the cloud is $\xi'_{int}(\gamma_{nt}) \triangleq r_n \cdot M_{int}/\mathbb{E}[G_{nt}] \cdot \gamma_{nt}$, where r_n is the energy used by the edge n per single-token transmission.

Second, the energy consumption of the cloud also includes the energy of computation and the energy of transmission. We only consider the former because the energy for transmitting the entire output sequence to the edge n depends on system inputs and is thus not the target of our optimization. For the inference request i from the edge n at the time slot t , due to the essential similarity between the forward pass and acceptance test operations and the prefill and decode operations, the computation energy can be expressed as $\psi_{int}(\gamma_{nt}) \triangleq M_{int}/\mathbb{E}[G_{nt}] \cdot (P(|\mathbf{h}_{i,n}| + \gamma_{nt})^2 + Q(|\mathbf{h}_{i,n}| + \gamma_{nt}))$, where P and Q are constants depending on the target LLM, the cloud energy efficiency, etc.

Energy Expense and Battery Management: The cloud-edge LLM inference infrastructure connects to a power grid; each edge also has its local battery or battery groups equipped, where the batteries can be charged or discharged dynamically. To support the LLM inference operations, an edge can draw energy from both the grid and its battery, and the cloud can draw energy from the grid. We denote the grid energy price at the time slot t as s_t , i.e., drawing grid energy to either charge the batteries or directly support LLM inference incurs energy expense. For the battery of the edge n at the time slot t , we denote its available battery energy as b_{nt} , enforcing $a_n \leq b_{nt} \leq c_n$, where a_n and c_n are the desired minimum and maximum battery limits, respectively, and can be specified by the system or the battery manufacturer. As every charge or discharge operation harms the battery’s lifespan, we consider the degradation [1] per unit energy charged or discharged as $d_n \triangleq m_n/\{2o_n(c_n - a_n)\}$, where m_n is the acquisition price of the battery and o_n is the maximum number of cycles that the battery can be operated for. We also use η_c and η_d to represent the battery charging and discharging efficiency. That is, if the battery of the edge n charges x_n^t and discharges y_n^t amount of energy, then the net charged and discharged energy is $x_n^t\eta_c$ and y_n^t/η_d . See Figure 3 for the time flow visualization.

Control Decisions: At each time slot t , the cloud-edge speculative-decoding-based LLM inference system makes the following control decisions: (i) $\gamma_{nt} \in \{1, \dots, \gamma_{\max}\}$, denoting the number of draft tokens to generate per iteration at the edge n with regard to all inference requests that arrive at the edge n ; (ii) $x_{nt} \geq 0$, denoting the amount of grid energy used to charge the battery of the edge n ; (iii) $y_{nt} \geq 0$, denoting the amount of energy discharged from the battery of the edge n to support the LLM inference on the edge n ; (iv) $z_{nt} \geq 0$,

来自边缘的请求 n 在时隙 t 。记 α 为云接受单个草稿令牌的概率。设 U_{nt} 为接受的草稿令牌数量, G_{nt} 为输出序列中的令牌数量。则我们有

$$\sum_{k=1}^{Y_{nt}} \alpha^k = 1 + \alpha \cdot \frac{1-\alpha^{Y_{nt}+1}}{1-\alpha} = \frac{1-\alpha^{Y_{nt}+1}}{1-\alpha}.$$

LLM 推理延迟: 我们考虑通过边缘与云之间的分布式投机解码在时间片 t 上解决推理请求所需的总体计算和通信延迟 i 。请注意, 将每个推理请求的上下文传输到云所产生的延迟和流量自然依赖于系统输入, 而不是本工作中考虑的控制决策, 下一步也将对此进行解释。

首先, 给定 M_{int} 作为输出序列的总长度, 即响应本次推理请求的输出标记总数, 本次推理请求的云 - 边缘推测解码迭代次数为 $M_{int}/\mathbb{E}[G_{nt}]$ 。

其次, 每次迭代包括如所述的步骤 $1\sim 4$, 对于每次这样的迭代, 我们以下内容: (i) 步骤 1 的计算延迟是 $\gamma_{nt}T'_n$, 其中 T'_n 表示在边缘自动回归生成单个标记的所有操作所花费的时间 n ; (ii) 步骤 2 的通信延迟是 $D_n + \beta_n\gamma_{nt}$, 其中 D_n 是边缘 n 与云之间的通信建立时间, β_n 是边缘 n 传输单个标记及其概率到云所花费的时间; (iii) 步骤 3 的计算延迟是 T'' , 这是云中单次并行前向传递以验证草稿标记所消耗的时间 (注意, 序列接受测试的时间通常与前向传递相比可以忽略不计); (iv) 步骤 4 的通信延迟是 $D_n + \beta'_n\mathbb{E}[G_{nt}]$, 其中 β'_n 是

因此, 总延迟, 写成 γ_{nt} 的函数, 是

$$\varphi_{int}(\gamma_{nt}) \triangleq M_{int}/\mathbb{E}[G_{nt}] \cdot T_{nt}, \quad (1)$$

其中 $T_{nt} = \gamma_{nt}T'_n + 2D_n + \beta_n\gamma_{nt} + T'' + \beta'_n\mathbb{E}[G_{nt}]$ 是每次迭代所有步骤的总延迟。

LLM 推理的流量: 对于基于推测解码的 LLM 推理, 每个来自 n 边缘的推理请求 i 在时间槽 t 时, 边缘 n 与云之间产生的网络流量, 可表示为 γ_{nt} 的函数为

$$\rho_{int}(\gamma_{nt}) \triangleq M_{int}/\mathbb{E}[G_{nt}] \cdot (\beta'_n\gamma_{nt} + \mathbb{E}[G_{nt}]), \quad (2)$$

其中 β'_n 指发送单个草稿令牌及其概率所产生的网络流量。为了防止消耗过多的通信资源, 我们对每个 t 的边缘 n 总累计流量施加预算 A_n 。

LLM 推理的能耗: 首先, 边缘的能耗包括计算能耗和传输能耗。对于在时间片 t 到达边缘的推理请求 i , 草稿 LLM 执行 “pre ll” 和 “decode” 操作以自回归地生成草稿标记, 这消耗了 $\xi'_{int}(\gamma_{nt}) \triangleq P_n \cdot |\mathbf{h}_{i,n}|^2 + Q_n \cdot |\mathbf{h}_{i,n}| \cdot M_{int}/\mathbb{E}[G_{nt}] \cdot \gamma_{nt}$ 的能量, 其中 P_n 和 Q_n 是取决于草稿 LLM、服务器能效等的常数; 传输所有草稿所需的能量

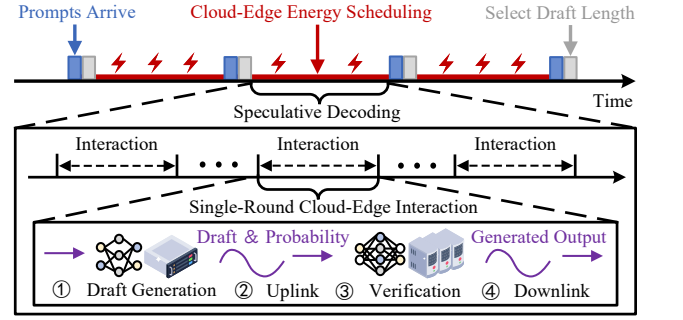


图 3: 云 - 边缘系统的时间流

将令牌传输到云端是 $\xi'_{int}(\gamma_{nt}) \triangleq r_n \cdot M_{int}/\mathbb{E}[G_{nt}] \cdot \gamma_{nt}$, 其中 r_n 是边缘 n 每次单令牌传输使用的能量。

其次, 云的能耗还包括计算能耗和传输能耗。我们只考虑前者, 因为将整个输出序列传输到边缘的能耗取决于系统输入, 因此不是我们优化的目标。对于时间槽的边缘推理请求, 由于前向传递和接受测试操作与预装载和解码操作在本质上相似, 计算能耗可以表示为, 其中和是取决于目标大语言模型、云能效等的常数。

能耗与电池管理: 云 - 边缘大型语言模型 (LLM) 推理基础设施连接到电网; 每个边缘节点还配备有本地电池或电池组, 这些电池可以动态充放电。为了支持 LLM 推理操作, 边缘节点可以从电网和自身电池中获取能量, 而云端则可以从电网获取能量。我们将时间段 t 的电网电价记为 s_t , 即从电网获取能量以充电或直接支持 LLM 推理都将产生能耗。对于时间段 t 的边缘电池 n , 我们将其可用电池能量记为 b_{nt} , 并执行 $a_n \leq b_{nt} \leq c_n$, 其中 a_n 和 c_n 分别为期望的最小和最大电池限制, 可由系统或电池制造商指定。由于每次充放电操作都会损害电池寿命, 我们将每单位能量充放电的降解视为 [1]。

控制决策: 在每个时间片 t 中, 云 - 边推测解码基础的大语言模型推理系统做出以下控制决策: (i) $\gamma_{nt} \in \{1, \dots, \gamma_{\max}\}$, 表示在边缘针对到达边缘的所有推理请求每次迭代生成的草稿令牌数量 n ; (ii) $x_{nt} \geq 0$, 表示用于为边缘电池充电的电网电量 n ; (iii) $y_{nt} \geq 0$, 表示从边缘电池释放的电量以支持边缘的大语言模型推理 n ; (iv) $z_{nt} \geq 0$,

¹The notation $\mathbb{E}[G_{nt}]$ could be $\mathbb{E}[G_{nt}|\alpha]$ to be more precise, as it is a conditional expectation for a given α .

¹符号 $\mathbb{E}[G_{nt}]$ 可以 $\mathbb{E}[G_{nt}|\alpha]$ 更加精确, 因为它是在给定 α 的条件下的期望值。

denoting the amount of grid energy used to support the LLM inference on the edge n ; (v) $e_t \geq 0$, denoting the amount of grid energy used to support the LLM inference in the cloud.

Total Cost of Cloud-Edge LLM Inference: The total cost of LLM inference contains multiple components as follows: (i) the total latency, i.e., $\sum_t \sum_i \sum_n \varphi_{int}(\gamma_{nt})$; (ii) the total energy expense of all edges, i.e., $\sum_t \sum_n s_t(x_{nt} + z_{nt})$; (iii) the total energy expense of the cloud, i.e., $\sum_t s_t e_t$; (iv) the degradation cost incurred by all edge batteries, i.e., $\sum_t \sum_n (d_n(x_{nt}\eta_c + y_{nt}/\eta_d))$.

2.2 Problem Formulation and Challenges

Problem Formulation: Using the models shown above, we formulate the optimization problem that controls the speculative decoding process to minimize the total cost:

$$\min \quad \mathcal{P} \triangleq \sum_t \sum_i \sum_n \varphi_{int}(\gamma_{nt}) + \sum_t (s_t(\sum_n (x_{nt} + z_{nt}) + e_t)) + \sum_t \sum_n (d_n(x_{nt}\eta_c + y_{nt}/\eta_d)) \quad (3)$$

$$\text{s.t.} \quad \sum_i \rho_{int}(\gamma_{nt}) \leq A_n, \forall t, \forall n, \quad (3a)$$

$$b_{nt} = b_{n,t-1} + x_{nt}\eta_c - y_{nt}/\eta_d, \forall t, \forall n, \quad (3b)$$

$$a_n \leq b_{nt} \leq c_n, \forall t, \forall n, \quad (3c)$$

$$\sum_i (\xi'_{int}(\gamma_{nt}) + \xi''_{int}(\gamma_{nt})) \leq y_{nt} + z_{nt}, \forall t, \forall n, \quad (3d)$$

$$\sum_i \sum_n \psi_{int}(\gamma_{nt}) \leq e_t, \forall t, \quad (3e)$$

$$\text{var.} \quad \gamma_{nt} \in \{1, \dots, \gamma_{\max}\}, x_{nt} \geq 0, y_{nt} \geq 0, z_{nt} \geq 0, e_t \geq 0, \forall t, \forall n. \quad (3f)$$

The objective (3) minimizes the total cost of LLM inference. Constraint (3a) ensures the network traffic remains within the budget. Constraint (3b) reflects the state transition of battery energy. Constraint (3c) applies the battery limits. Constraint (3d) ensures the power grid and the edge battery jointly offer sufficient energy to each edge to support LLM inference. Constraint (3e) ensures the power grid offers sufficient energy to the cloud to support LLM inference. Constraint (3f) specifies the domains of decision variables.

Problem Challenges: It is not trivial to solve the problem $\mathcal{P}$ online.² First, $\mathcal{P}$ is online optimization with time-coupled state transitions. Constraints (3b) and (3c) impose a decisive relation upon the control decisions x_{nt} and y_{nt} across time slots. Every decision made currently based on current inputs will inevitably impact what decisions can be made in the future; however, future inputs remain uncertain currently, making anticipatory decision-making hard. Second, many inputs, including $\varphi_{int}(\cdot)$, to $\mathcal{P}$ are posterior, i.e., only revealed at t after a concrete control action is taken at t , which escalates the difficulty of online decision-making. Third, the mixed-integer nonlinearity makes $\mathcal{P}$ intractable even in an offline setting, not to mention we desire to solve it in an online manner. Overall, this problem is difficult to solve because of its discrete decisions, online uncertainty, and time-coupled constraints.

3 Online Algorithms Design

3.1 Decomposed Scheduling: Algorithm 1

Algorithmic Rationale: Our core idea is decomposing our original problem $\mathcal{P}$ into two subproblems $\mathcal{P}_1$ and $\mathcal{P}_2$, and then addressing the *integrality* and *posterior*, contained in $\mathcal{P}_1$, and the *state transitions* and *online uncertainty*, contained in $\mathcal{P}_2$, separately. Based on

²We use the notation like $\mathcal{P}$ to refer to either an optimization problem or just its objective function, which is clear from the context.

Algorithm 1 Energy-Aware Online Scheduling Framework

```

1: Initialize: Set  $x_{n0} = y_{n0} = z_{n0} = e_0 = 0$ ;
2: for  $t = 1, 2 \dots T$  do
3:   Invoke Algorithm 2 to get draft length  $\gamma_{nt}$ ;
4:   Given draft length  $\gamma_{nt}$ , invoke Algorithm 3 to get  $e_t$ 
     for cloud energy and  $\{x_{nt}, y_{nt}, z_{nt}\}$  for edge energy;
5:   Execute speculative decoding on the cloud and edges;
6: end for
```

this idea, we design Algorithm 1, which we call the Energy-Aware One-Line Scheduling framework (EAOS), as our overall control algorithm. In each time slot, Algorithm 1 invokes Algorithms 2 and 3 to address $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively, which will be elaborated in the next two subsections, and performs the speculative-decoding-based LLM inference following the control decisions made.

Problem Decomposition: We derive the two subproblems.

The first subproblem $\mathcal{P}_1$ is as follows:

$$\min \quad \mathcal{P}_1 \triangleq \sum_t \sum_i \sum_n \varphi_{int}(\gamma_{nt}) \quad (4)$$

$$\text{s.t.} \quad \sum_i \rho_{int}(\gamma_{nt}) \leq A_n, \forall t, \forall n, \quad (4a)$$

$$\text{var.} \quad \gamma_{nt} \in \{1, \dots, \gamma_{\max}\}, \forall t, \forall n. \quad (4b)$$

The objective (4) is the total latency. Constraint (4a) originates from (3a). Constraint (4b) specifies the domain for the draft length.

The second subproblem $\mathcal{P}_2$ is as follows:

$$\min \quad \mathcal{P}_2 \triangleq \sum_t (s_t(\sum_n (x_{nt} + z_{nt}) + e_t)) + \sum_t \sum_n (d_n(x_{nt}\eta_c + y_{nt}/\eta_d)) \quad (5)$$

$$\text{s.t.} \quad \sum_t g_t \triangleq \sum_t (T - t)(x_{nt}\eta_c - y_{nt}/\eta_d) - T(c_n - a_n) \leq 0, \forall n, \quad (5a)$$

$$h_{1t} \triangleq \sum_i (\xi'_{int}(\bar{\gamma}_{nt}) + \xi''_{int}(\bar{\gamma}_{nt})) - y_{nt} - z_{nt} \leq 0, \forall t, \forall n, \quad (5b)$$

$$h_{2t} \triangleq \sum_i \sum_n \psi_{int}(\bar{\gamma}_{nt}) - e_t \leq 0, \forall t, \quad (5c)$$

$$\text{var.} \quad e_t \geq 0, x_{nt} \geq 0, y_{nt} \geq 0, z_{nt} \geq 0, \forall t, \forall n. \quad (5d)$$

The objective (5) captures the energy expense and the degradation cost of edge batteries. Constraint (5a) is derived from the combination of (3b) and (3c), that is, $b_{n,T+1} = b_{nT} + x_{nT}\eta_c - y_{nT}/\eta_d = b_{n,T-1} + \sum_{t=T-1}^T (x_{nt}\eta_c - y_{nt}/\eta_d) = \dots = b_{n0} + \sum_{t=1}^T (x_{nt}\eta_c - y_{nt}/\eta_d)$, $\forall n$, which leads to $\sum_{t=1}^t (x_{nt}\eta_c - y_{nt}/\eta_d) = b_{n,t+1} - b_{n0} \leq c_n - a_n, \forall n$. Summing them from $t = 1$ to $t = T - 1$, we obtain Constraint (5a). Constraints (5b) and (5c) are respectively transformed from (3d) and (3e), where $\bar{\gamma}_{nt}$, as solved in the problem $\mathcal{P}_1$,³ is applied as input. The decision variables of energy scheduling are listed in (5d).

Remarks and Insights: (i) In the objective of $\mathcal{P}$, the draft length γ_{nt} affects latency through the expected output token count $\mathbb{E}[G_{nt}] = \frac{1-\alpha^{\gamma_{nt}+1}}{1-\alpha}$; in the constraints of $\mathcal{P}$, the traffic and energy also depend on γ_{nt} through $\mathbb{E}[G_{nt}]$. This motivates us to leverage it as a bridge between the two subproblems in our algorithm design. (ii) Given the aforementioned challenges for $\mathcal{P}$, we use decomposition to prioritize online tractability and operational feasibility over absolute optimality, while rigorously bounding the resulting sub-optimality. Section 4 offers the performance analysis, bounding

³For decisions, we use the notation like γ_{nt} to refer to the decision *variable*, and the notation like $\bar{\gamma}_{nt}$, i.e., decorated symbol, to refer to a specific *value* that the corresponding decision variable takes.

表示用于支持边缘 LLM 推理的电网能量的数量 n ; (v) $e_t \geq 0$,

表示用于支持云端 LLM 推理的电网能量的数量。

云 - 边缘大语言模型推理的总成本: LLM 推理的总成本包含以下多个组成部分: (i) 总延迟, 即 $\sum_t \sum_i \sum_n \varphi_{int}(\gamma_{nt})$; (ii) 所有边缘节点的总能耗, 即 $\sum_t \sum_n s_t(x_{nt} + z_{nt})$; (iii) 云端的总能耗, 即 $\sum_t s_t e_t$; (iv) 所有边缘电池的损耗成本, 即 $\sum_t \sum_n (d_n(x_{nt}\eta_c + y_{nt}/\eta_d))$.

2.2 问题的提出与挑战

问题表述: 使用上述模型, 我们制定了控制推测解码过程的优化问题, 以最小化总成本:

$$\min \quad \mathcal{P} \triangleq \sum_t \sum_i \sum_n \varphi_{int}(\gamma_{nt}) + \sum_t (s_t(\sum_n (x_{nt} + z_{nt}) + e_t)) + \sum_t \sum_n (d_n(x_{nt}\eta_c + y_{nt}/\eta_d)) \quad (3)$$

$$\text{s.t.} \quad \sum_i \rho_{int}(\gamma_{nt}) \leq A_n, \forall t, \forall n, \quad (3a)$$

$$b_{nt} = b_{n,t-1} + x_{nt}\eta_c - y_{nt}/\eta_d, \forall t, \forall n, \quad (3b)$$

$$a_n \leq b_{nt} \leq c_n, \forall t, \forall n, \quad (3c)$$

$$\sum_i (\xi'_{int}(\gamma_{nt}) + \xi''_{int}(\gamma_{nt})) \leq y_{nt} + z_{nt}, \forall t, \forall n, \quad (3d)$$

$$\sum_i \sum_n \psi_{int}(\gamma_{nt}) \leq e_t, \forall t, \quad (3e)$$

$$\text{变量.} \quad \gamma_{nt} \in \{1, \dots, \gamma_{\max}\}, x_{nt} \geq 0, y_{nt} \geq 0, z_{nt} \geq 0, e_t \geq 0, \forall t, \forall n. \quad (3f)$$

目标 (3) 最小化 LLM 推理的总成本。约束 (3a) 确保网络流量保持在预算范围内。约束 (3b) 反映电池能量的状态转移。约束 (3c) 适用于电池限制。约束 (3d) 确保电网和边缘电池共同为每个边缘提供足够的能量以支持 LLM 推理。约束 (3e) 确保电网为云端提供足够的能量以支持 LLM 推理。约束 (3f) 指定决策变量的域。

问题挑战: 在线解决该问题并非易事。² 首先, $\mathcal{P}$ 是具有一时间耦合状态转换的在线优化。约束条件 (3b) 和 (3c) 在不同时隙之间对控制决策 x_{nt} 和 y_{nt} 施加了决定性关系。当前基于当前输入所做的每个决策都会不可避免地影响未来能够做出的决策; 然而, 未来的输入目前仍不确定, 这使得前瞻性决策变得困难。其次, 许多输入, 包括 $\varphi_{int}(\cdot)$, 到 $\mathcal{P}$ 是后验的, 即只有在 t 采取具体控制措施后才被揭示, 这增加了在线决策的难度。第三, 混合整数非线性使得 $\mathcal{P}$ 即使在离线环境下也难以处理, 更不用说我们希望以在线方式解决它了。总体而言, 由于该问题具有离散决策、在线不确定性和时间耦合约束, 因此难以解决。

3 在线算法设计

3.1 分解调度: 算法 1

算法原理: 我们的核心思想是将我们原始的问题 $\mathcal{P}$ 分解为两个子问题 $\mathcal{P}_1$ 和 $\mathcal{P}_2$, 然后分别处理包含在 $\mathcal{P}_1$ 中的整体性和后验, 以及包含在 $\mathcal{P}_2$ 中的状态转移和在线不确定性。基于

Algorithm 1 Energy-Aware Online Scheduling Framework

```

1: Initialize: Set  $x_{n0} = y_{n0} = z_{n0} = e_0 = 0$ ;
2: for  $t = 1, 2 \dots T$  do
3:   Invoke Algorithm 2 to get draft length  $\gamma_{nt}$ ;
4:   Given draft length  $\gamma_{nt}$ , invoke Algorithm 3 to get  $e_t$ 
     for cloud energy and  $\{x_{nt}, y_{nt}, z_{nt}\}$  for edge energy;
5:   Execute speculative decoding on the cloud and edges;
6: end for
```

基于这一想法, 我们设计了算法 1, 我们称之为能量感知在线调度框架 (EAOS), 作为我们的整体控制算法。在每个时间片中, 算法 1 调用算法 2 和算法 3 来分别解决 $\mathcal{P}_1$ 和 $\mathcal{P}_2$ 问题, 这将在接下来的两个小节中详细说明, 并根据所做的控制决策执行基于推测解码的 LLM 推理。

问题分解: 我们导出了两个子问题。

第一个子问题 $\mathcal{P}_1$ 如下:

$$\min \quad \mathcal{P}_1 \triangleq \sum_t \sum_i \sum_n \varphi_{int}(\gamma_{nt}) \quad (4)$$

$$\text{s.t.} \quad \sum_i \rho_{int}(\gamma_{nt}) \leq A_n, \forall t, \forall n, \quad (4a)$$

$$\text{var.} \quad \gamma_{nt} \in \{1, \dots, \gamma_{\max}\}, \forall t, \forall n. \quad (4b)$$

目标 (4) 是总延迟。约束 (4a) 来源于 (3a)。约束 (4b) 指定了草稿长度的范围。

第二个子问题 $\mathcal{P}_2$ 如下:

$$\min \quad \mathcal{P}_2 \triangleq \sum_t (s_t(\sum_n (x_{nt} + z_{nt}) + e_t)) + \sum_t \sum_n (d_n(x_{nt}\eta_c + y_{nt}/\eta_d)) \quad (5)$$

$$\text{s.t.} \quad \sum_t g_t \triangleq \sum_t (T - t)(x_{nt}\eta_c - y_{nt}/\eta_d) - T(c_n - a_n) \leq 0, \forall n, \quad (5a)$$

$$h_{1t} \triangleq \sum_i (\xi'_{int}(\bar{\gamma}_{nt}) + \xi''_{int}(\bar{\gamma}_{nt})) - y_{nt} - z_{nt} \leq 0, \forall t, \forall n, \quad (5b)$$

$$h_{2t} \triangleq \sum_i \sum_n \psi_{int}(\bar{\gamma}_{nt}) - e_t \leq 0, \forall t, \quad (5c)$$

$$\text{var.} \quad e_t \geq 0, x_{nt} \geq 0, y_{nt} \geq 0, z_{nt} \geq 0, \forall t, \forall n. \quad (5d)$$

目标 (5) 反映了边缘电池的能量消耗和退化成本。约束 (5a) 是由 (3b) 和 (3c) 的组合推导而来的, 即 $b_{n,T+1} = b_{nT} + x_{nT}\eta_c - y_{nT}/\eta_d = b_{n,T-1} + \sum_{t=T-1}^T (x_{nt}\eta_c - y_{nt}/\eta_d) = \dots = b_{n0} + \sum_{t=1}^T (x_{nt}\eta_c - y_{nt}/\eta_d), \forall n$, 这导致 $\sum_{t=1}^t (x_{nt}\eta_c - y_{nt}/\eta_d) = b_{n,t+1} - b_{n0} \leq c_n - a_n, \forall n$ 。从 $t = 1$ 到 $t = T - 1$ 求和, 我们得到约束 (5a)。约束 (5b) 和 (5c) 分别由 (3d) 和 (3e) 转化而来, 其中 $\bar{\gamma}_{nt}$ 作为在问题 $\mathcal{P}_1$ ³ 中求解的结果, 被用作输入。能量调度的决策变量列于 (5d) 中。

备注与见解: (i) 在 $\mathcal{P}$ 的目标中, 草稿长度 γ_{nt} 通过预期输出 token 数量 $\mathbb{E}[G_{nt}] = \frac{1-\alpha^{\gamma_{nt}+1}}{1-\alpha}$ 影响延迟; 在 $\mathcal{P}$ 的约束条件下, 流量和能量也依赖于 γ_{nt} 通过 $\mathbb{E}[G_{nt}]$ 。这促使我们在算法设计中将其作为两个子问题之间的桥梁。(ii) 鉴于上述对 $\mathcal{P}$ 的挑战, 我们使用分解方法优先考虑在线可处理性和操作可行性, 而非绝对最优性, 同时严格界定由此产生的次优性。第 4 节提供了性能分析及界定。

³对于决策, 我们使用类似 γ_{nt} 的符号来表示决策变量, 以及类似 $\bar{\gamma}_{nt}$ 的符号, 即带修饰的符号, 来表示对应决策变量所取的特定值。

²我们使用类似 $\mathcal{P}$ 的符号来指代优化问题本身或只是它的目标函数, 这可以从上下文中清楚地看出。

Algorithm 2 Draft Length Control Algorithm**Input:** Inference requests $i \in \mathcal{I}_t = \{1, \dots, I_t\}$, $I = \max I_t$.**Output:** Selected draft length $\tilde{y}_{nt}$.

```

1: Initialize: weight  $v_{n1}^j = 1$  for  $\forall j \in \{1, \dots, y_{\max}\}$ ,  $\forall n$ ;
2: for  $\forall t, \forall n$  do
3:   Construct feasible set  $\mathcal{F}_{nt} = \{j : \sum_i \rho_{int}(y_{nt}^j) \leq A_n\}$ ;

4:   Compute probability  $p_{nt}^j = (1 - \varsigma) \frac{v_{nt}^j}{\sum_{j \in \mathcal{F}_{nt}} v_{nt}^j} + \frac{\varsigma}{|\mathcal{F}_{nt}|}$ ;

5:   Sample arm  $\bar{j}$  from  $p_{nt}^j$ ,  $\forall j \in \mathcal{F}_{nt}$ , then  $\tilde{y}_{nt} \leftarrow \bar{j}$ ;
6:   Observe latency  $\varphi_{1nt}(y_{nt}^{\bar{j}}), \dots, \varphi_{I_t nt}(y_{nt}^{\bar{j}})$ 
7:   Update estimator  $\ell_{nt}^{\bar{j}} = \frac{\sum_{i \in \mathcal{I}_t} \varphi_{int}(y_{nt}^{\bar{j}})}{p_{nt}^{\bar{j}}}$ ,  $\ell_{nt}^j = 0$  for  $j \neq \bar{j}$ ;

8:   Update weight  $v_{n,t+1}^j = v_{nt}^j \exp(-\frac{\varsigma}{I_{Y_{\max}}} \ell_{nt}^j)$ .
9: end for
```

such sub-optimality, and serves as the theoretical evidence that the decomposition-based approach works.

3.2 Control of Draft Length: Algorithm 2

For the problem $\mathcal{P}_1$, as the number of users’ prompts varies over time and Constraint (4a) limits the feasible draft length options, we propose to design *adversarial bandit learning* in Algorithm 2. Unlike standard adversarial bandit learning where only one *reward* is disclosed after one *arm* is selected, in our case here, we have a varying number of multiple rewards (due to processing multiple inference requests) after one draft length option is chosen. We will need to handle this non-trivial difference in our algorithm.

Algorithm 2 addresses the bandit problem for each edge n separately, and treats different draft lengths as different arms while maintaining and updating a set of weights v_{nt}^j for each arm j as time goes. In Line 1, all arm weights are initialized to 1 to reflect no prior preference among arms. For each edge n , at each time slot t , in Line 3, we first construct a feasible set $\mathcal{F}_{nt}$ that includes only those arms whose cumulative traffic footprint across all prompts does not exceed the edge’s budget. In Line 4, based on the weights, we compute a probability distribution p_{nt}^j over $\mathcal{F}_{nt}$, which is a mixture of an exploitation term proportional to the weights and a uniform exploration term scaled by a parameter ς . This exploration-exploitation trade-off is essential in adversarial environments where the inference processing latency patterns may change unpredictably. Afterwards, we sample an arm $\bar{j}$ according to p_{nt}^j in Line 5, observe the resulting latencies $\varphi_{int}(y_{nt}^{\bar{j}})$ for all prompts in Line 6, and construct an unbiased estimator $\ell_{nt}^{\bar{j}}$ by dividing the total observed latency by the selection probability in Line 7. This importance-weighted estimator is crucial for handling the partial feedback nature of the bandit problem, where only the latency of the selected arm is observed, yet we need to estimate the performance of all arms. Finally, in Line 8, the weights of all the arms are updated using exponential weighting, where the magnitude of updating is scaled by $\varsigma/(I_{Y_{\max}})$ to ensure stability. Arms that yield higher latency receive larger estimated losses, causing their weights to decrease exponentially, making them less likely to be selected in future time slots. The time complexity of Algorithm 2 is $O(IN_{Y_{\max}})$.

Remarks and Insights: (i) Our method enforces Constraint (4a) by restricting the action space to $\mathcal{F}_{nt}$, which eliminates constraint

Algorithm 3 Energy Control Algorithm**Input:** $f_t(\cdot), g_t(\cdot), h_{1t}(\cdot), h_{2t}(\cdot), \omega_t = \{x_{nt}, y_{nt}, z_{nt}, e_t\}$.**Output:** $\omega_{t+1} = \{x_{n,t+1}, y_{n,t+1}, z_{n,t+1}, e_{t+1}\}$.

```

1: Initialize:  $\omega_0 = 0, \lambda_0 = 0, g_0(\cdot) = 0$ ;
2: Update decision  $\omega_{t+1}$  according to (6);
3: Update virtual queues  $\lambda_{n,t+1}$  according to (7).
```

violations without parameter tuning and makes the feasibility check computationally light. (ii) We have successfully handled the setting where each arm selection yields multiple rewards by aggregating such rewards through inverse probability weighting to obtain a single unbiased estimator for the chosen arm.

3.3 Control of Energy: Algorithm 3

For the problem $\mathcal{P}_2$, to facilitate our algorithm design, we donate $\omega_t \triangleq \{x_{nt}, y_{nt}, z_{nt}, e_t\}$ as the control variables. We decompose $\mathcal{P}_2$ into one-shot problems $\mathcal{P}_{2,t}$, $\forall t$ and solve it with a primal-dual-based method that progresses online without future inputs.

At any time slot t , we first solve the following one-shot problem and assign its solution to ω^{t+1} :

$$\min \mathcal{P}_{2,t} \triangleq f_t(\omega) + (\lambda_t + g_{t-1}(\omega_t))^\top g_t(\omega) + \mu \|\omega - \omega_t\|_2^2 \quad (6)$$

$$\text{s.t. } h_{1t}(\omega_t) \leq 0, h_{2t}(\omega_t) \leq 0, \forall t, \quad (6a)$$

$$\text{var. } \omega_t \in \mathbb{W}, \forall t, \quad (6b)$$

where ω_t is the primal decision variable and λ_t is the dual decision variable. The parameter μ is a positive step size, and the term $\mu \|\omega - \omega_t\|_2^2$ is a regularization and proximal term. Next, we construct a “virtual queue” that is updated as

$$\lambda_{n,t+1} = \max\{-g_{nt}(\omega_{t+1}), \lambda_{nt} + g_{nt}(\omega_{t+1})\}, \forall t, \forall n. \quad (7)$$

Here, $\lambda_t = [\lambda_{1t}, \lambda_{2t}, \dots, \lambda_{Nt}]^\top$ and $g_t = [g_{1t}, g_{2t}, \dots, g_{Nt}]^\top$. If we substitute the initial term inside the maximization with zero, the update for $\lambda_{n,t+1}$ becomes equivalent to conventional queue updates using the increments $g_{nt}(\omega_{t+1})$. Consequently, we can find that $\lambda_{n,t+1}, \forall t, \forall n$ serve to quantify the accumulated constraint violations and that the bounds for the queue backlogs can be directly converted into corresponding bounds for the constraint violations.

Algorithm 3 adopts the above idea. In such alternate steps, i.e., updating ω_{t+1} in (6) and λ_{t+1} in (7), we only need the inputs before (excluding) $t + 1$ rather than any future information. $\mathcal{P}_{2,t}$ can be solved through standard convex optimization solvers, such as the interior-point method [35]. The time complexity of Algorithm 3 is thus $O(N^2 \log(1/\sigma))$ for calculating a σ -accurate solution.

Remarks and Insights: (i) Our step size μ is designed independently of the time horizon length T , making it suitable for scenarios where the total length of the entire time horizon is unknown in advance and the online optimization may terminate at any arbitrary time slot. (ii) When computing ω_t for t , no information at or after t is needed, making the computation process entirely on the fly.

4 Theoretical Performance Analysis

We first study the theoretical guarantees that come with our Algorithms 2 and 3 that solve $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively. Then, we combine them to obtain the performance of our entire approach, i.e., Algorithm 1, which solves the original problem $\mathcal{P}$. Note that

Algorithm 2 Draft Length Control Algorithm**Input:** Inference requests $i \in \mathcal{I}_t = \{1, \dots, I_t\}$, $I = \max I_t$.**Output:** Selected draft length $\tilde{y}_{nt}$.

```

1: Initialize: weight  $v_{n1}^j = 1$  for  $\forall j \in \{1, \dots, y_{\max}\}$ ,  $\forall n$ ;
2: for  $\forall t, \forall n$  do
3:   Construct feasible set  $\mathcal{F}_{nt} = \{j : \sum_i \rho_{int}(y_{nt}^j) \leq A_n\}$ ;

4:   Compute probability  $p_{nt}^j = (1 - \varsigma) \frac{v_{nt}^j}{\sum_{j \in \mathcal{F}_{nt}} v_{nt}^j} + \frac{\varsigma}{|\mathcal{F}_{nt}|}$ ;

5:   Sample arm  $\bar{j}$  from  $p_{nt}^j$ ,  $\forall j \in \mathcal{F}_{nt}$ , then  $\tilde{y}_{nt} \leftarrow \bar{j}$ ;
6:   Observe latency  $\varphi_{1nt}(y_{nt}^{\bar{j}}), \dots, \varphi_{I_t nt}(y_{nt}^{\bar{j}})$ 
7:   Update estimator  $\ell_{nt}^{\bar{j}} = \frac{\sum_{i \in \mathcal{I}_t} \varphi_{int}(y_{nt}^{\bar{j}})}{p_{nt}^{\bar{j}}}$ ,  $\ell_{nt}^j = 0$  for  $j \neq \bar{j}$ ;

8:   Update weight  $v_{n,t+1}^j = v_{nt}^j \exp(-\frac{\varsigma}{I_{Y_{\max}}} \ell_{nt}^j)$ .
9: end for
```

这种次优性，并作为基于分解的方法有效性的理论证据。

3.2 草稿长度控制：算法 2

对于问题 $\mathcal{P}_1$ ，由于用户提示的数量随时间变化，并且约束条件 (4a) 限制了可行的草稿长度选项，我们提出在算法 2 中设计对抗性赌博机学习。与标准的对抗性赌博机学习在选择一个臂后只揭示一个奖励不同，在我们的情况中，选择一个草稿长度选项后会产生多个奖励（由于处理多个推理请求）且数量不固定。我们需要在算法中处理这一非平凡的差异。

算法 2 分别针对每条边解决赌博机问题，并将不同的草稿长度视为不同的臂，同时维护并更新每个臂的一组权重随时间变化。在第 1 行，所有臂的权重初始化为 1，以反映臂之间没有先验偏好。对于每条边，在每个时间段，在第 3 行，我们首先构建一个可行集合，其中仅包含那些在所有提示中累积流量不超过该边预算的臂。在第 4 行，根据权重，我们计算一个概率分布，该概率分布是一个利用项（与权重成比例）和由参数缩放的均匀探索项的混合。在对抗环境中，这种探索 - 利用权衡是至关重要的，因为推理处理延迟模式可能会不可预测地变化。随后，我们根据该分布抽样一个臂

Algorithm 3 Energy Control Algorithm**Input:** $f_t(\cdot), g_t(\cdot), h_{1t}(\cdot), h_{2t}(\cdot), \omega_t = \{x_{nt}, y_{nt}, z_{nt}, e_t\}$.**Output:** $\omega_{t+1} = \{x_{n,t+1}, y_{n,t+1}, z_{n,t+1}, e_{t+1}\}$.

```

1: Initialize:  $\omega_0 = 0, \lambda_0 = 0, g_0(\cdot) = 0$ ;
2: Update decision  $\omega_{t+1}$  according to (6);
3: Update virtual queues  $\lambda_{n,t+1}$  according to (7).
```

在没有参数调优的情况下进行违规处理，并且使可行性检查在计算上轻量化。(ii) 我们已经成功处理了每次臂选择产生多个奖励的情况，通过逆概率加权将这些奖励聚合，以获得所选臂的单一无偏估计。

3.3 能量控制：算法 3

对于问题 $\mathcal{P}_2$ ，为了方便我们的算法设计，我们将

$\omega_t \triangleq \{x_{nt}, y_{nt}, z_{nt}, e_t\}$ 作为控制变量。我们将 $\mathcal{P}_2$ 分解为一次性问题 $\mathcal{P}_{2,t}$, $\forall t$ ，并使用基于原始 - 对偶的方法在线求解，无需未来输入。

在任何时间段 t ，我们首先求解以下一次性问题，并将其解分配给 ω^{t+1} :

$$\min \mathcal{P}_{2,t} \triangleq f_t(\omega) + (\lambda_t + g_{t-1}(\omega_t))^\top g_t(\omega) + \mu \|\omega - \omega_t\|_2^2 \quad (6)$$

$$\text{s.t. } h_{1t}(\omega_t) \leq 0, h_{2t}(\omega_t) \leq 0, \forall t, \quad (6a)$$

$$\text{var. } \omega_t \in \mathbb{W}, \forall t, \quad (6b)$$

其中 ω_t 是原始决策变量， λ_t 是对偶决策变量。参数 μ 是一个正的步长，而项 $\mu \|\omega - \omega_t\|_2^2$ 是正则化和近端项。接下来，我们构建一个“虚拟队列”，其更新方式为

$$\lambda_{n,t+1} = \max\{-g_{nt}(\omega_{t+1}), \lambda_{nt} + g_{nt}(\omega_{t+1})\}, \forall t, \forall n. \quad (7)$$

这里， $\lambda_t = [\lambda_{1t}, \lambda_{2t}, \dots, \lambda_{Nt}]^\top$ 和 $g_t = [g_{1t}, g_{2t}, \dots, g_{Nt}]^\top$ 。如果我们将最大化中初始项替换为零， $\lambda_{n,t+1}$ 的更新将等同于使用增量 $g_{nt}(\omega_{t+1})$ 的传统队列更新。因此，我们可以发现 λ_n , $t+1, \forall t, \forall n$ 用于量化累积的约束违规，并且队列积压的界限可以直接转换为约束违规的相应界限。

算法 3 采用了上述思想。在这样的交替步骤中，即在 (6) 中更新 ω_{t+1} 和在 (7) 中更新 λ_{t+1} ，我们只需要在 $t + 1$ 之前（不包括 $t + 1$ ）的输入，而不需要任何未来信息。 $\mathcal{P}_{2,t}$ 可以通过标准凸优化求解器来解决，例如内点法 [35]。因此，算法 3 的时间复杂度为 $O(N^2 \log(1/\sigma))$ 以计算一个 σ 精度的解。

备注与见解： (i) 我们的步长 μ 是独立于时间范围长度 T 设计的，使其适用于总时间范围长度事先未知且在线优化可能任意时间终止的场景。(ii) 在计算 ω_t 对于 t 时，不需要 t 及之后的任何信息，从而使计算过程完全依赖于 y 。

4 理论性能分析

我们首先研究随我们的算法 2 和算法 3 所带来的理论保证，这两个算法分别解决 $\mathcal{P}_1$ 和 $\mathcal{P}_2$ 。然后，我们将它们结合起来以获得我们整体方法的性能，即算法 1，它解决原始问题 $\mathcal{P}$ 。注意

备注与见解：（i）我们的方法通过将动作空间限制为 $\mathcal{F}_{nt}$ 来执行约束（4a），从而消除了约束

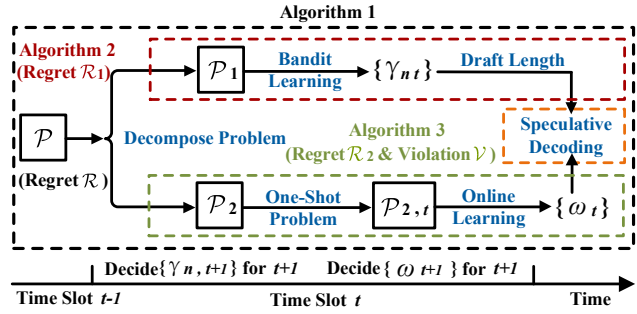


Figure 4: Roadmap for theoretical analysis

such combination analysis is non-trivial. The roadmap for such theoretical analysis is shown in Figure 4.

For the problem $\mathcal{P}_1$, the *regret* captures the gap between the expected objective value evaluated with the online solutions from our bandit-learning-based approach and the objective value evaluated with the optimum that selects the best arm for the entire time horizon in hindsight. We prove that such regret accumulated over time is sub-linear with regard to the length of the time horizon T , i.e., it grows slower than the natural passage of time.

THEOREM 4.1. *The regret of $\mathcal{P}_1$ is upper-bounded as*

$$\mathcal{R}_1 \triangleq \mathbb{E}[\mathcal{P}_1(\bar{y}_{nt})] - \mathcal{P}_1(y_n^*) \leq 2IN \sqrt{2T\gamma_{\max} \ln \gamma_{\max}},$$

where $\bar{y}_{nt}$ is the online solution obtained from Algorithm 2 and y_n^* is the static time-invariant offline optimal decision for $\mathcal{P}_1$ over the whole time horizon.

PROOF. See Appendix A. $\square$

For the problem $\mathcal{P}_2$, the *regret* quantifies the gap between the objective value gained by our primal-dual-based approach and the sum of the optimal objective values obtained from solving each corresponding one-shot problem independently. We prove the sub-linear growth of the regret. The *constraint violation* captures the cumulative violation of Constraint (5a) over time. That is, this long-term constraint defined for the entire time horizon (in contrast to any instantaneous constraint defined for each single time slot) is generally hard to satisfy; therefore, we allow its violation but prove such violation is also sub-linear, i.e., our approach is asymptotically feasible and the cumulative violation is decaying effectively.

THEOREM 4.2. *The regret of $\mathcal{P}_2$ is upper-bounded as*

$$\mathcal{R}_2 \triangleq \mathcal{P}_2(\bar{\omega}_t) - \mathcal{P}_2(\omega_t^*) \leq o(T),$$

where $\bar{\omega}_t$ is the online solution obtained from Algorithm 3, and ω_t^* is the per-slot offline optimal decision for $\mathcal{P}_2$ given $\bar{y}_{nt}$ at time slot t .

PROOF. See Appendix B. $\square$

THEOREM 4.3. *The constraint violation of $\mathcal{P}_2$ is upper-bounded as*

$$\mathcal{V} \triangleq \sum_t g_t(\bar{\omega}_t) \leq o(T),$$

where $\bar{\omega}_t$ is the online solution obtained from Algorithm 3.

PROOF. See Appendix C. $\square$

For the problem $\mathcal{P}$, regarding its *regret*, we choose to compare the expected objective value incurred by our entire online approach, i.e., Algorithm 1, against that incurred by the static offline optimum, i.e., the single best draft length and energy policy chosen with full hindsight for the entire time horizon. This is a standard practice in existing research. Yet, note that simply summing the outcomes of Theorems 4.1 and 4.2 is not valid, since their respective optima are defined in distinct senses. Instead, establishing Theorem 4.4 necessitates breaking down the overall regret into several parts, with auxiliary terms to facilitate and complete the reasoning.

THEOREM 4.4. *The static regret of $\mathcal{P}$ is upper-bounded as*

$$\mathcal{R} \triangleq \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathcal{P}(\hat{y}_n, \hat{\omega}) \leq o(T),$$

where $\bar{y}_{nt}, \bar{\omega}_t$ are the online decisions obtained from our algorithms, and $\hat{y}_n, \hat{\omega}$ are the joint static time-invariant offline optimal decisions for $\mathcal{P}$ over the whole time horizon.

PROOF. See Appendix D. $\square$

Remarks and Insights: (i) *Comparators:* For the subproblem $\mathcal{P}_1$, the comparator is the static offline optimal solution y_n^* . For the subproblem $\mathcal{P}_2$, the comparator is the series of the per-slot optimal solution ω_t^* given the online solution $\bar{y}_{nt}$ at each t from Algorithm 2. For the original problem $\mathcal{P}$, the comparator is the static offline optimal solution $\hat{y}_n, \hat{\omega}$. (ii) *Coupling:* In Appendix D, to upper-bound the full regret $\mathcal{R}$, we decompose it into three gaps: the first two are bounded by exploiting the sub-linear regrets $\mathcal{R}_1$ and $\mathcal{R}_2$; the third is automatically non-positive. Consequently, $\mathcal{R}$ is also sub-linear. (iii) *Alternative:* Note that for $\mathcal{P}$, an alternative comparator can be the dynamic offline optimum, i.e., the solution to the full time-coupled problem $\mathcal{P}$ with perfect future knowledge and time-varying decisions. The gap between the aforementioned static comparator and such a dynamic comparator could be generally linear in T without additional restrictive assumptions, which we intend to investigate further in the future.

5 Experimental Evaluations

5.1 Experimental Settings

Datasets and LLMs: We adopt well-known datasets spanning diverse task categories, including the translation task (IWLST17-de-en [25], WMT14-de-en [26]), the summarization task (CNN/Daily [27]), and the question-answering task (SQuAD v2 [28], TruthfulQA [29]). Specifically, the model pairs are (LLaMA-68M, LLaMA-7B), (LLaMA-1.1B, LLaMA-7B), and (LLaMA-1.1B, LLaMA-13B), which are widely used for evaluating speculative decoding [7, 8]. The maximum draft length $\gamma_{\max}$ is set as 20 tokens commonly [7].

Cloud-Edge System: We use the Australian EUA dataset [32], containing 125 edge nodes and 816 mobile users in the Melbourne central business district (CBD) area. We select the location with the highest density of mobile users as the site for the cloud server. The cloud is equipped with NVIDIA A800 PCIe 80GB, and each edge is equipped with NVIDIA GeForce RTX 4090 24GB. We consider the time horizon $T = 300$ and each user sends 0~5 inference requests at each time slot, where a single time slot is set to 1 minute [36]. The number of inference requests that each user submits to her edge is set in proportion to the dynamic job arrival trace of Google clusters [30]. Based on user density, the traffic parameter β_n'' is set as 2~4 B,

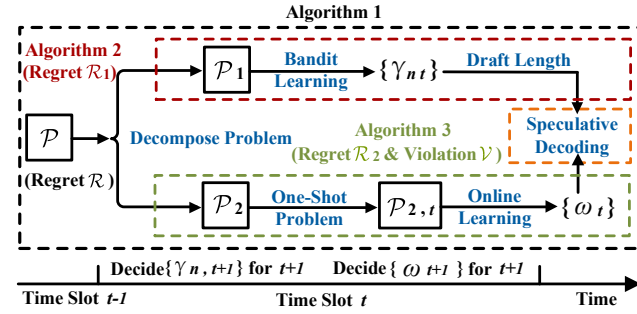


图 4: 理论分析路线图

这种组合分析并非平凡。此类理论分析的路线图如图 4 所示。

对于问题 $\mathcal{P}_1$ ，遗憾衡量了基于我们基于赌博机学习方法
的在线解评估的期望目标值与事后选择整个时间范围内最佳
臂所评估的最优目标值之间的差距。我们证明，这种随时间
积累的遗憾与时间范围 T 的长度相比是次线性的，即它的增
长速度比时间的自然流逝要慢。

定理 4.1. $\mathcal{P}_1$ 的遗憾上界为

$$\mathcal{R}_1 \triangleq \mathbb{E}[\mathcal{P}_1(\bar{y}_{nt})] - \mathcal{P}_1(y_n^*) \leq 2IN \sqrt{2T\gamma_{\max} \ln \gamma_{\max}},$$

其中 $\bar{y}_{nt}$ 是由算法 2 获得的在线解， y_n^* 是针对 $\mathcal{P}_1$ 在整个
时间范围内的静态时间不变的离线最优决策。

证明。见附录 A. $\square$

对于问题 $\mathcal{P}_2$ ，遗憾值量化了我们基于原始 - 对偶方法获
得的目标值与独立求解每个对应一次性问题所得最优目标值
之和之间的差距。我们证明了遗憾值的次线性增长。约束违
反捕捉了约束 (5a) 随时间的累计违反情况。也就是说，这个
为整个时间范围定义的长期约束（与为每个单独时间段定义
的瞬时约束相对）通常难以满足；因此，我们允许其违反，
但也证明这种违反是次线性的，即我们的方法在渐近上是可
行的，且累计违反正在有效下降。

定理 4.2. $\mathcal{P}_2$ 的遗憾上界为

$$\mathcal{R}_2 \triangleq \mathcal{P}_2(\bar{\omega}_t) - \mathcal{P}_2(\omega_t^*) \leq o(T),$$

其中 $\bar{\omega}_t$ 是从算法 3 得到的在线解， ω_t^* 是在时间槽 t 给定 $\bar{y}_{nt}$ 时，对 $\mathcal{P}_2$
的每槽离线最优决策。

证明。见附录 B. $\square$

定理 4.3. $\mathcal{P}_2$ 的约束违反上界为

$$\mathcal{V} \triangleq \sum_t g_t(\bar{\omega}_t) \leq o(T),$$

其中 $\bar{\omega}_t$ 是从算法 3 获得的在线解。

证明。见附录 C. $\square$

对于问题 $\mathcal{P}$ ，关于其遗憾，我们选择将整个在线方法
（即算法 1）产生的期望目标值与静态离线最优（即在整个
时间跨度内凭完全回顾选择的单一最佳草案长度和能量策略）
产生的期望目标值进行比较。这是现有研究中的标准做法。
然而，请注意，简单地将定理 4.1 和 4.2 的结果相加是不成立
的，因为它们各自的最优值是以不同的意义定义的。相反，
要建立定理 4.4，需要将整体遗憾分解为若干部分，并引入辅
助项以便于和完成推理。

定理 4.4. $\mathcal{P}$ 的静态遗憾上界为

$$\mathcal{R} \triangleq \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathcal{P}(\hat{y}_n, \hat{\omega}) \leq o(T),$$

其中 $\bar{y}_{nt}, \bar{\omega}_t$ 是从我们的算法获得的在线决策， $\hat{y}_n, \hat{\omega}$ 是针对
 $\mathcal{P}$ 在整个时间范围内的联合静态时间不变离线最优决策。

PROOF. See Appendix D. $\square$

备注与见解: (i) 比较对象：对于子问题 $\mathcal{P}_1$ ，比较对象
是静态离线最优解 y_n^* 。对于子问题 $\mathcal{P}_2$ ，比较对象是在每个 t
时刻，给定在线解 $\bar{y}_{nt}$ 的每个时间段最优解序列 ω_t^* ，来源于
算法 2。对于原始问题 $\mathcal{P}$ ，比较对象是静态离线最优解 $\hat{y}_n, \hat{\omega}$ 。
(ii) 耦合：在附录 D 中，为了上界完整的遗憾值 $\mathcal{R}$ ，我们
将其分解为三个差距：前两个通过利用子线性遗憾 $\mathcal{R}_1$ 和 $\mathcal{R}_2$ 来
进行界定；第三个差距自动为非正。因此， $\mathcal{R}$ 也是子线性的。
(iii) 替代方案：注意对于 $\mathcal{P}$ ，一个替代比较对象可以是动态
离线最优解，即在具有完美未来知识和时间变化决策的情况
下，针对完整的时间耦合问题 $\mathcal{P}$ 的解。此前

5 实验评估

5.1 实验设置

数据集与大型语言模型（LLMs）：我们采用涵盖多种任务类别
的知名数据集，包括翻译任务（IWLST17-de-en [25], WMT14-
de-en [26]）、摘要任务（CNN/ 每日 [27]）以及问答任务
（SQuAD v2 [28], TruthfulQA[29]）。具体而言，模型对为
（LLaMA-68M, LLaMA-7B）、（LLaMA-1.1B, LLaMA-7B）
和（LLaMA-1.1B, LLaMA-13B），这些模型对被广泛用于评估
推测解码 [7, 8]。最大草稿长度 $\gamma_{\max}$ 通常设置为 20 个标记 [7]。

云 - 边缘系统：我们使用澳大利亚 EUA 数据集 [32]，其中
包含位于墨尔本中央商务区（CBD）地区的 125 个边缘节点
和 816 名移动用户。我们选择移动用户密度最高的位置作为云服
务器的部署地点。云端配备了 NVIDIA A800 PCIe 80GB，每个
边缘节点配备了 NVIDIA GeForce RTX 4090 24GB。我们考虑
时间范围 $T = 300$ ，每个用户在每个时间片发送 0~5 推理请求，
其中单个时间片设置为 1 分钟 [36]。每个用户提交到其边缘节点
的推理请求数量按 Google 集群的动态作业到达轨迹 [30] 进行比
例设置。基于用户密度，流量参数 β_n'' 设置为 2~4B，

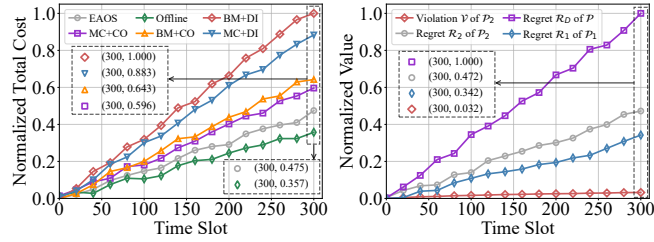


Figure 5: Total cost

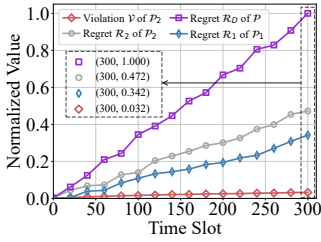
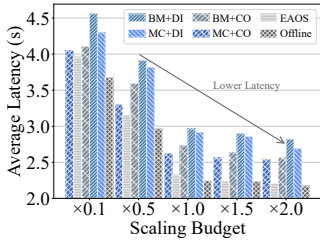
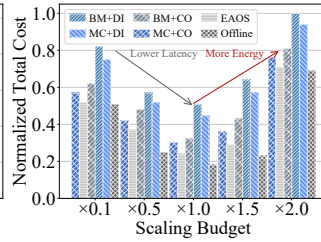


Figure 6: Regret and violation

Figure 7: Impact of A_n on $\mathcal{P}_1$ Figure 8: Impact of A_n on $\mathcal{P}$

and the budget A_n is set as 55~65 MB [33]. The time parameter β_n is set as 0.19~0.35 ms, β'_n set as 0.12~0.24 ms [1], T'_n set as 1.7~2.4 ms, and T'' set as 2.2~3.8 ms [15]. The energy parameter r_n is set as $4.2\sim 4.8\times 10^{-10}$ J, P_n set as 0.08~0.15 J, Q_n set as 0.11~0.30 J, P set as 0.27~0.83 J, and Q set as 0.55~1.32 J based on server efficiency and LLMs, etc [37~39]. We consider lithium-ion batteries as edge batteries, where the charging efficiency η_c is set as 99.9% and the discharging efficiency η_d is set as 85% [9]. The battery capacity at each edge is set to support the peak power demand for 12 hours [11]. The battery degradation parameter d_n is set as 8.3×10^{-5} \$/Wh [34]. We set the energy price s_t using the real-world trace of South Australia during October 26-27, 2021 [31].

Baselines: For draft length selection, we consider BMOP [40]: A state-of-the-art approach utilizing bandit learning with a probing budget; MCEW [41]: A state-of-the-art approach using bandit play with a operating budget. For cloud-edge energy scheduling, we consider COMU [42]: A state-of-the-art approach applying lower-and-upper-bounded virtual queues for online learning; DINU [43]: A state-of-the-art approach adopting gradient sampling and online mirror ascent for optimization. We combine these methods and denote all the combined approaches as (i) BM+CO; (ii) BM+DI; (iii) MC+CO; (iv) MC+DI. And we have the following offline solution: (v) Offline: The dynamic offline optimum (i.e., optimal solution varying over time) that solves the problem $\mathcal{P}$ as a whole via the Gurobi [44] optimization solver with all inputs provided in advance. We then compare our proposed approach EAOS with these baselines.

Here, we highlight that we intentionally omit the static offline optimum from our experiments because it is a strictly weaker benchmark than the dynamic offline optimum. The dynamic optimum is the absolute lower bound and a strictly stronger comparator. Any algorithm that performs well against this benchmark will necessarily outperform the static optimum. Including the static line would add visual clutter without providing additional insight into our algorithm's adaptability. Our theory already proves sub-linear static regret as a worst-case guarantee; the experiments are designed to test our algorithms' practical performance against the hardest possible comparator. This dual approach ensures both worst-case robustness (theory) and practical excellence (experiments).

5.2 Evaluation Results

All plots in Figures 5~16 are measured from our experiments.

Total Cost: Figure 5 shows the total cost of the cloud-edge LLM inference system. Our proposed approach EAOS consistently outperforms other alternative approaches and is the closest to the offline optimum. The total cost of our method is ultimately decreased by

26.1% compared to BM+CO, by 52.5% compared to BM+DI, by 20.3% compared to MC+CO, and by 46.2% compared to MC+DI.

Regret and Violation: Figure 6 presents the regret $\mathcal{R}_1$ of the problem $\mathcal{P}_1$, the regret $\mathcal{R}_2$ and the constraint violation $\mathcal{V}$ of the problem $\mathcal{P}_2$, and the gap $\mathcal{R}_D$ between the objective value incurred by our online solutions and that incurred by the dynamic offline optimal solutions for the problem $\mathcal{P}$. We observe that $\mathcal{R}_1$ and $\mathcal{R}_2$ are smaller than $\mathcal{R}_D$. We also observe that the constraint violation is one order of magnitude less than the regrets.

Impact of Budget: Figure 7 and Figure 8 exhibit the impact of the communication budget A_n in the problem $\mathcal{P}_1$ and the problem $\mathcal{P}$. As A_n increases, longer draft lengths become feasible, allowing the draft model to be configured with more appropriate draft lengths to reduce the total latency. Yet, as A_n continues to increase, the cloud-edge system consumes more energy to obtain lower latency, leading to an increase in the total cost. We find that our method dynamically adjusts decisions and consistently outperforms other approaches under various restrictions on A_n .

Impact of Workload: Figure 9 visualizes the impact of the edge workload on the total cost. As the number of the LLM inference requests submitted by each user to the edge increases, each edge needs to handle more speculative decoding interactions with the cloud. The total delay and total energy consumption will both increase as a result. Our proposed method demonstrates greater stability at large scales compared to other alternative approaches.

Latency Weight: Figure 10 demonstrates the impact of the total latency weight on the total cost. The total cost includes both latency (measured in seconds) and energy overhead (measured in dollars), and the cloud-edge system can adjust the weights between these two factors to achieve different optimization priorities. We observe that our proposed approach continues to show superior performance across varying total latency weight settings.

Execution Time: Figure 11 reflects the execution time of Algorithm 2 and Algorithm 3 (the latency of speculative decoding is shown in Figures 14 and 15 below). It can be observed that both of them are able to complete within tens of milliseconds, with a gradual increase as the workload grows. This is fully acceptable compared to the response time of typical LLM services in reality.

Impact of Parameter μ : Figure 12 depicts the impact of the parameter μ which makes Algorithm 3 more accurate but adds computation in the process of operation. As shown, as μ increases, the total cost declines sharply and the runtime rises slowly. However, once μ increases beyond a certain value, further increases will cause the total cost to rise, whereas the runtime grows substantially.

Bandit Learning: Figure 13 details the bandit learning. Without violating the communication traffic constraints, Algorithm 2

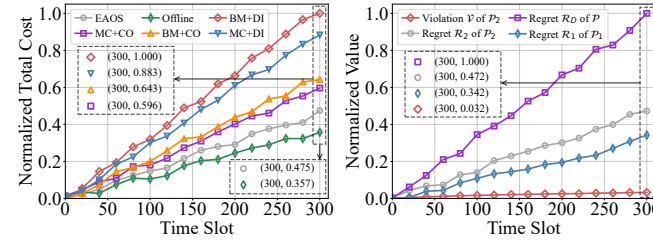


Figure 5: Total cost

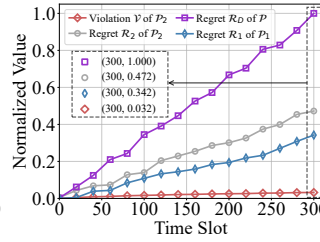
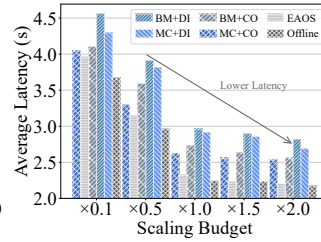
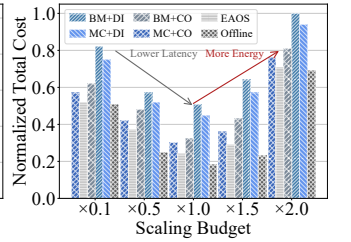


图 6: 遗憾与违规

图 7: A_n 对 $\mathcal{P}_1$ 的影响图 8: A_n 对 $\mathcal{P}$ 的影响

预算 A_n 设定为 55~65 MB[33]。时间参数 β_n 设定为 0.19~0.35 毫秒, β'_n 设定为 0.12~0.24 毫秒 [1], T'_n 设定为 1.7~2.4 毫秒, T'' 设定为 2.2~3.8 毫秒 [15]。能量参数 r_n 设定为 $4.2\sim 4.8\times 10^{-10}$ 焦耳, P_n 设定为 0.08~0.15 焦耳, Q_n 设定为 0.11~0.30 焦耳, P 设定为 0.27~0.83 焦耳, 且 Q 设定为 0.55~1.32 焦耳, 基于服务器效率和大语言模型等 [3739]。我们将锂离子电池视为边缘电池, 其中充电效率 η_c 设定为 99.9%, 放电效率 η_d 设定为 85%[9]。每个边缘的电池容量设定为支持峰值功率需求 12 小时 [11]。电池退化参数 d_n 设定为 8.3×10^{-5} \$/Wh[34]。我们使用南澳大利亚在十月的实际数据轨迹设置能源价格 s_t 。

基线方法: 在草稿长度选择方面, 我们考虑 BMOP [40]: 一种利用带有探测预算的赌博学习的最先进方法; MCEW [41]: 一种使用带有操作预算的赌博策略的最先进方法。在云 - 边缘能源调度方面, 我们考虑 COMU [42]: 一种应用上下界虚拟队列进行在线学习的最先进方法; DINU [43]: 一种采用梯度采样和在在线镜像上升进行优化的最先进方法。我们将这些方法结合起来, 并将所有组合方法表示为 (i) BM+CO ; (ii) BM+DI ; (iii) MC+CO ; (iv) MC+DI。我们还有以下离线解决方案: (v) 离线: 动态离线最优解 (即随时间变化的最优解), 通过 Gurobi [44] 优化求解器在所有输入均已提前提供的情况下整体解决问题 $\mathcal{P}$ 。然后我们将提出的方法 EAOS 与这些基线方法进行比较。

在这里, 我们强调我们有意在实验中省略静态离线最优解, 因为它是一个严格较弱的基准, 而动态离线最优解则是绝对的下界和严格更强的比较对象。任何在该基准上表现良好的算法必然会超越静态最优解。包括静态线只会增加视觉上的杂乱, 而不会提供关于我们算法适应性的额外见解。我们的理论已经证明了在最坏情况下静态后悔是次线性的; 实验旨在测试我们的算法在面对最困难的比较对象时的实际性能。这种双重方法确保了最坏情况的鲁棒性 (理论) 和实际卓越性 (实验)。

5.2 评估结果

图 5~16 中的所有图表均来自我们的实验测量。

总成本: 图 5 显示了云边 LLM 推理系统的总成本。我们提出的方法 EAOS 持续优于其他替代方法, 并且最接近离线最优。我们方法的总成本最终降低了

与 BM+CO 相比为 26.1%, 与 BM+DI 相比为 52.5%, 与 MC+CO 相比为 20.3%, 与 MC+DI 相比为 46.2%。

后悔与违规: 图 6 展示了问题 $\mathcal{P}_1$ 的后悔 $\mathcal{R}_1$, 问题 $\mathcal{P}_2$ 的后悔 $\mathcal{R}_2$ 和约束违规 $\mathcal{V}$, 以及我们的在线解所产生的目标值与问题 $\mathcal{P}$ 的动态离线最优解所产生的目标值之间的差距 $\mathcal{R}_D$ 。我们观察到 $\mathcal{R}_1$ 和 $\mathcal{R}_2$ 小于 $\mathcal{R}_D$ 。我们还观察到约束违规比后悔低一个数量级。

预算影响: 图 7 和图 8 展示了通信预算在问题 $\mathcal{P}_1$ 和问题 $\mathcal{P}$ 中的影响。随着 A_n 的增加, 更长的草稿长度变得可行, 使草稿模型可以配置更合适的草稿长度以减少总延迟。然而, 随着 A_n 的继续增加, 云 - 边缘系统为了获得更低的延迟而消耗更多的能量, 从而导致总成本增加。我们发现我们的方法能够动态调整决策, 并在不同的 A_n 限制下始终优于其他方法。

工作负载的影响: 图 9 展示了边缘工作负载对总成本的影响。随着每个用户提交到边缘的 LLM 推理请求数量增加, 每个边缘节点需要处理更多与云端的推测解码交互。因此, 总延迟和总能耗都会增加。我们提出的方法在大规模情况下表现出比其他替代方法更高的稳定性。

延迟权重: 图 10 展示了总延迟权重对总成本的影响。总成本包括延迟 (以秒为单位) 和能耗开销 (以美元为单位), 云 - 边缘系统可以在这两个因素之间调整权重以实现不同的优化优先级。我们观察到, 在不同的总延迟权重设置下, 我们提出的方法仍然显示出优越的性能。

执行时间: 图 11 反映了算法 2 和算法 3 的执行时间 (推测解码的延迟见下方图 14 和图 15)。可以观察到, 它们都能在几十毫秒内完成, 随着工作负载的增加而逐渐增长。与现实中小型大型语言模型服务的响应时间相比, 这是完全可以接受的。

参数的影响 μ : 图 12 显示了参数 μ 的影响, 该参数使算法 3 更准确, 但在操作过程中增加了计算量。如图所示, 随着 μ 的增加, 总成本急剧下降, 而运行时间缓慢增加。然而, 一旦 μ 超过某个值, 进一步增加将导致总成本上升, 同时运行时间显著增长。

盗贼学习: 图 13 详细说明了盗贼学习。在不违反通信流量约束的情况下, 算法 2

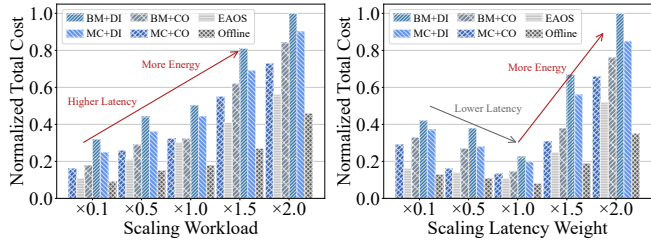


Figure 9: Impact of workload

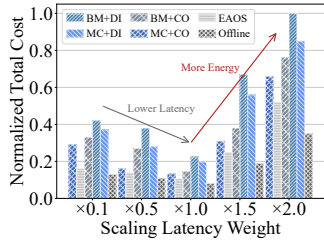


Figure 10: Latency weight

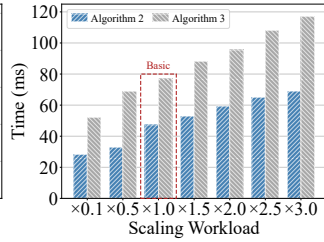


Figure 11: Execution time

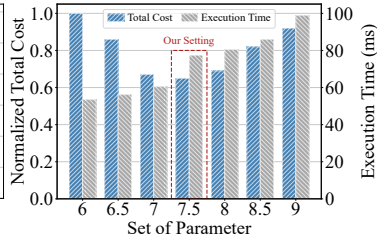
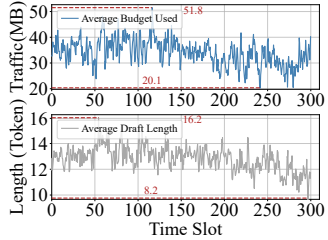
Figure 12: Impact of parameter μ 

Figure 13: Bandit learning

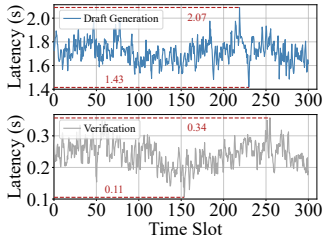


Figure 14: Comp. latency

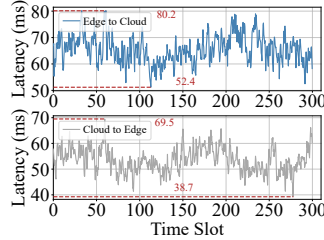


Figure 15: Comm. latency

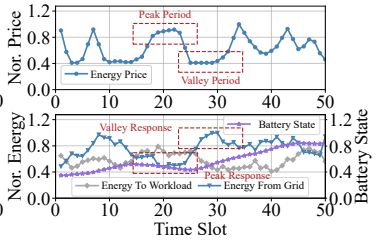


Figure 16: Battery management

strategically selects the draft length to rationally allocate communication resources across each time slot, ensuring that each inference request can be processed via speculative decoding effectively. And we have marked the maximum and minimum values in the figure.

Details of Latency: Figure 14 reflects the average computational latency of speculative decoding. As mentioned earlier, the computational latency includes the processing time at both the edge and the cloud. Figure 15 reveals the average communication latency of speculative decoding. Also, as previously stated, the communication latency includes the time for bidirectional communication between the edge and the cloud. And we have marked the maximum and minimum values of the latency in the Figures 14 and 15.

Battery Management: Figure 16 illustrates the scheduling of edge battery operations. We find that when the energy price from the power grid is at its peak or valley period, the energy from the edge batteries to the workload, and the energy from the grid to the edge batteries, will respond in a regular pattern timely. Correspondingly, the total battery state will also change regularly according to the energy price. This response mechanism can effectively utilize battery buffering to provide continuous energy to the LLM services.

6 Related Work

We review existing research in different groups, and for each group, highlight the insufficiency compared to our work, respectively.

LLM Inference Services with Cloud-Edge Collaboration: He *et al.* [4] study LLM task offloading and resource allocation in cloud-edge settings via active inference. Ma *et al.* [12] address LLM scheduling and data flow in cloud-edge environments. Yao *et al.* [1] improve LLM service quality and response time using multi-agent reinforcement learning within a vector database assisted cloud-edge framework. Ren *et al.* [13] investigate LLM inference offloading and computational load reduction by combining reinforcement learning with decentralized identity management. Huang *et al.* [14] optimize LLM prompt security, latency, and resource use in cloud-edge systems using a multi-stage dynamic Bayesian game model.

This group of works exploit cloud-edge collaboration to enable and provision LLM inference services, but fail to investigate speculative decoding, which is a concrete technique to significantly accelerate LLM inference. They also typically lack considerations of edge battery operations for sustainable LLM services.

Speculative Decoding in Resource-Limited Environments: Xu *et al.* [15] accelerate constrained token generation with adaptive draft branch navigation and pipelined verification. Ye *et al.* [16] optimize constrained prefill and autoregressive stages via a carefully-designed speculative decoding method. Svirschevski *et al.* [17] enable high-throughput decoding on memory-limited hardware via a draft token tree. Ning *et al.* [5] cut communication overhead in device-edge collaboration by split verification. Bhattacharjee *et al.* [6] transmit draft token distributions across constrained computing nodes via sparsification and lattice quantization.

These studies focus on speculative decoding, yet often ignore the aspect of dynamic energy consumption and management in the speculative decoding process. Moreover, they often do not take a rigorous formal approach like ours to model, analyze, and optimize the holistic system performance in an online fashion.

Battery-Aware Energy Management in Edge Computing: Zheng *et al.* [9] propose a deep reinforcement learning based approach in coping with the dynamic power demand for edge-cloud collaboration. Deng *et al.* [18] study stochastic task and energy model in wireless-powered mobile edge computing systems. Meng *et al.* [19] design a learning-driven method for decentralized model training in wireless edge computing systems to overcome limited resources and battery life. Li *et al.* [20] focus on pattern-aware online energy optimization to extend the lifespan of edge batteries. Wang *et al.* [10] transform edge AI infrastructures into virtual power plants via smart scheduling of edge battery energy.

Although these investigations cover edge batteries, they often overlook LLM inference, let alone the energy complexity caused by interactions between the cloud and the edges due to speculative decoding. Our work, to the best of our knowledge, is the first to incorporate battery-energy scheduling in LLM speculative decoding.

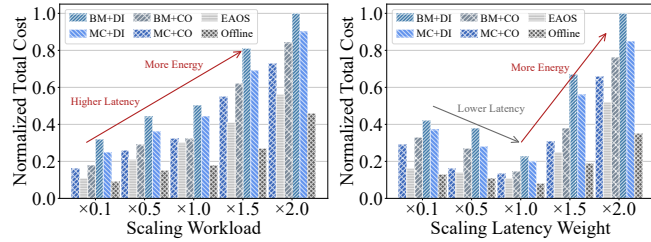


Figure 9: Impact of workload

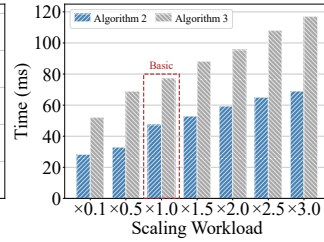


Figure 10: Latency weight

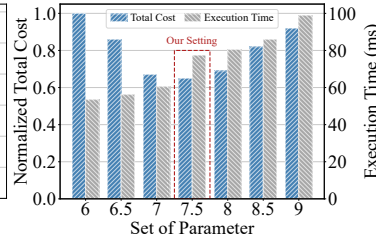


图 11: 执行时间

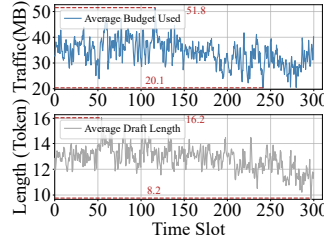
图 12: 参数 μ 的影响

图 13: 盗匪学习

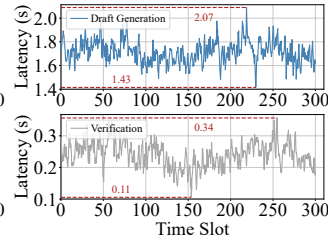


图 14: 计算延迟

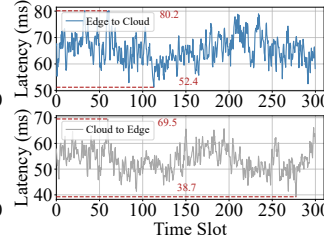


图 15: 通信延迟

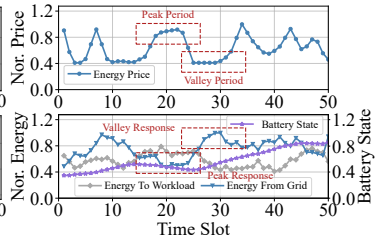


图 16: 电池管理

战略性地选择草稿长度，以合理分配每个时间片的通信资源，确保每个推理请求能够通过推测解码有效处理。我们已经在图中标出了最大值和最小值。

延迟详情: 图 14 反映了推测解码的平均计算延迟。如前所述，计算延迟包括边缘和云端的处理时间。图 15 显示了推测解码的平均通信延迟。同样，如前所述，通信延迟包括边缘与云之间双向通信的时间。在图 14 和图 15 中，我们还标出了延迟的最大值和最小值。

电池管理: 图 16 展示了边缘电池操作的调度。我们发现，当电网的电价处于峰值或谷值期间时，边缘电池向工作负载提供的能量，以及电网向边缘电池提供的能量，将会按规律及时响应。相应地，电池总状态也会根据电价规律性变化。这种响应机制可以有效利用电池缓冲，为大型语言模型（LLM）服务提供持续的能量。

6 相关工作

我们回顾了不同群体中现有的研究，并针对每个群体，分别强调其与我们的工作相比的不足之处。

具有云 - 边协作的 LLM 推理服务: He 等人 [4] 通过主动推理研究了云 - 边环境中的 LLM 任务卸载和资源分配。Ma 等人 [12] 解决了云 - 边环境中的 LLM 调度和数据流问题。Yao 等人 [1] 在向量数据库辅助的云 - 边框架内，利用多智能体强化学习提高了 LLM 服务质量和响应时间。Ren 等人 [13] 通过将强化学习与分布式身份管理结合，研究了 LLM 推理卸载和计算负载减少。Huang 等人 [14] 使用多阶段动态贝叶斯博弈模型优化了云 - 边系统中 LLM 提示的安全性、延迟和资源使用。

这一组工作利用云 - 边协作来启用和提供大语言模型推理服务，但未能研究推测性解码，这是一种能够显著加速大语言模型推理的具体技术。它们通常也缺乏对边缘电池运行以实现可持续大语言模型服务的考量。

资源有限环境中的投机解码: Xu 等人 [15] 通过自适应草稿分支导航和流水线验证加速受限的标记生成。Ye 等人 [16] 通过精心设计的投机解码方法优化受限的预填充和自回归阶段。Svirschevski 等人 [17] 通过草稿标记树在内存有限的硬件上实现高吞吐量解码。Ning 等人 [5] 通过分割验证减少设备边缘协作中的通信开销。Bhattacharjee 等人 [6] 通过稀疏化和格量化在受限计算节点之间传输草稿标记分布。

这些研究关注于推测解码，但往往忽略了推测解码过程中动态能耗和管理的方面。此外，他们通常没有像我们这样采用严格的形式方法在线建模、分析和优化整体系统性能。

边缘计算中的电池感知能源管理: 郑等人 [9] 提出了一种基于深度强化学习的方法，以应对边缘 - 云协作中动态电力需求。邓等人 [18] 研究了无线供能移动边缘计算系统中的随机任务和能量模型。孟等人 [19] 设计了一种基于学习的方法，用于无线边缘计算系统中分散式模型训练，以克服资源有限和电池寿命限制。李等人 [20] 聚焦于模式感知的在线能耗优化，以延长边缘电池的使用寿命。王等人 [10] 通过智能调度边缘电池能量，将边缘人工智能基础设施转化为虚拟电厂。

虽然这些研究涵盖了边缘电池，但它们常常忽略了大型语言模型（LLM）的推理，更不用说由于推测解码而导致的云与边缘之间交互的能量复杂性。据我们所知，我们的工作为首个将电池能量调度纳入 LLM 推测解码的研究。

7 Conclusion and Future Work

Distributed speculative decoding can greatly speed up LLM inference in cloud-edge environments, but a comprehensive treatment of its delay, traffic, and energy management, especially in battery-powered mobile edge computing settings, has been largely missing. This paper fills this gap through novel analytical modeling, algorithm design, theoretical analysis, and practical experiments, focusing on inventively integrating bandit learning and online learning for system optimization in an online manner in the long run.

Future work could include multiple research directions. We intend to design incentive mechanisms and security protocols for speculative-decoding-based cloud-edge collaborative LLM inference, and also study the optimization of other types of LLM inference acceleration techniques across cloud and edge boundaries.

Acknowledgments

This work was supported in part by the Science and Technology Projects of Xizang Autonomous Region (XZ202401ZY0027), the Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, the Beijing Natural Science Foundation (2026FTZD001 and 4222033), the Beijing Key Laboratory of Multimodal Data Intelligent Perception and Governance, and the U.S. National Science Foundation (CNS-2047719 and CNS-2225949).

References

- [1] Zhi Yao, Zhiqing Tang, Wenmian Yang, and Weijia Jia. 2025. Enhancing LLM QoS Through Cloud-Edge Collaboration: A Diffusion-Based Multi-Agent Reinforcement Learning Approach. *IEEE Transactions on Services Computing* 18, 3 (2025), 1412–1427.
- [2] Yandi Li, Jianxiong Guo, Zhiqing Tang, Xingjian Ding, Juncheng Wang, Tian Wang, and Weijia Jia. 2025. Cloud-Edge System for Scheduling Unpredictable LLM Requests With Combinatorial Bandit. *IEEE Transactions on Services Computing* 18, 6 (2025), 3567–3580.
- [3] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *ICML*.
- [4] Ying He, Jingcheng Fang, F. Richard Yu, and Victor C. Leung. 2024. Large Language Models (LLMs) Inference Offloading and Resource Allocation in Cloud-Edge Computing: An Active Inference Approach. *IEEE Transactions on Mobile Computing* 23, 12 (2024), 11253–11264.
- [5] JIAHONG NING, Ce ZHENG, and Tingting Yang. 2025. DSSD: Efficient Edge-Device Deployment and Collaborative Inference via Distributed Split Speculative Decoding. In *ICML ML4Wireless Workshop*.
- [6] Payel Bhattacharjee, Fengwei Tian, Meiyu Zhong, Guangyi Zhang, Osvaldo Simeone, and Ravi Tandon. 2025. Conformal Sparsification for Bandwidth-Efficient Edge–Cloud Speculative Decoding. In *NeurIPS AI4NextG Workshop*.
- [7] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *ACM ASPLOS*.
- [8] Fahao Chen, Peng Li, Tom H. Luan, Zhou Su, and Jing Deng. 2025. SPIN: Accelerating Large Language Model Inference with Heterogeneous Speculative Models. In *IEEE INFOCOM*.
- [9] Dongyu Zheng, Lei Liu, Guoming Tang, Yi Wang, and Weichao Li. 2024. Power Demand Reshaping Using Energy Storage for Distributed Edge Clouds. *IEEE Transactions on Parallel and Distributed Systems* 35, 2 (2024), 362–376.
- [10] Hengdi Wang, Lei Jiao, Konglin Zhu, Lin Zhang, and Jin Dong. 2026. Multi-Tenant Edge AI as a Virtual Power Plant via Online Auction-Driven Energy Scheduling. *IEEE Transactions on Mobile Computing* (2026).
- [11] Guoming Tang, Hao Yuan, Deke Guo, Kui Wu, and Yi Wang. 2021. Reusing Backup Batteries as BESS for Power Demand Reshaping in 5G and Beyond. In *IEEE INFOCOM*.
- [12] Mulei Ma, Chenyu Gong, Liekang Zeng, and Yang Yang. 2025. Multi-Tier Multi-Node Scheduling of LLM for Collaborative AI Computing. In *IEEE INFOCOM*.
- [13] Yuzheng Ren, Haijun Zhang, Fei Richard Yu, Wei Li, Pincan Zhao, and Ying He. 2025. Industrial Internet of Things With Large Language Models (LLMs): An Intelligence-Based Reinforcement Learning Approach. *IEEE Transactions on Mobile Computing* 24, 5 (2025), 4136–4152.

- [14] Haiyang Huang, Tianhui Meng, and Weijia Jia. 2025. Joint Optimization of Prompt Security and System Performance in Edge-Cloud LLM Systems. In *IEEE INFOCOM*.
- [15] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2025. EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding. *IEEE Transactions on Mobile Computing* 24, 4 (2025), 3256–3273.
- [16] Shengyuan Ye, Bei Ouyang, Liekang Zeng, Tianyi Qian, Xiaowen Chu, Jian Tang, and Xu Chen. 2025. Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices. In *IEEE INFOCOM*.
- [17] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. 2024. SpecExec: massively parallel speculative decoding for interactive LLM inference on consumer devices. In *NeurIPS*.
- [18] Xiumei Deng, Jun Li, Long Shi, Zhiqiang Wei, Xiaobo Zhou, and Jinhong Yuan. 2022. Wireless Powered Mobile Edge Computing: Dynamic Resource Allocation and Throughput Maximization. *IEEE Transactions on Mobile Computing* 21, 6 (2022), 2271–2288.
- [19] Zeyu Meng, Hongli Xu, Min Chen, Yang Xu, Yangming Zhao, and Chunming Qiao. 2021. Learning-Driven Decentralized Machine Learning in Resource-Constrained Wireless Edge Computing. In *IEEE INFOCOM*.
- [20] Qing Li, Shangguang Wang, Xiao Ma, Ao Zhou, Yue Wang, Gang Huang, and Xuanzhe Liu. 2024. Battery-Aware Energy Optimization for Satellite Edge Computing. *IEEE Transactions on Services Computing* 17, 2 (2024), 437–451.
- [21] Ilai Bistritz, Zhengyuan Zhou, Xi Chen, Nicholas Bambos, and Jose Blanchet. 2019. Online exp3 learning in adversarial bandits with delayed feedback. In *NeurIPS*.
- [22] Taishi Uchiya, Atsuyoshi Nakamura, and Mineichi Kudo. 2010. Algorithms for adversarial bandit problems with multiple plays. In *ALT*.
- [23] Eric C Hall and Rebecca M Willett. 2015. Online convex optimization in dynamic environments. *IEEE Journal of Selected Topics in Signal Processing* 9, 4 (2015), 647–662.
- [24] Tianyi Chen, Qing Ling, and Georgios B Giannakis. 2017. An online convex optimization approach to proactive network resource allocation. *IEEE Transactions on Signal Processing* 65, 24 (2017), 6350–6364.
- [25] Mauro Cettolo et al. 2017. Overview of the IWSLT 2017 Evaluation Campaign. In *IWSLT*.
- [26] Ondřej Bojar et al. 2014. Findings of the 2014 Workshop on Statistical Machine Translation. In *ACL SMT Workshop*.
- [27] Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get to the point: Summarization with pointer-generator networks. In *ACL*.
- [28] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. *arXiv preprint arXiv:1606.05250* (2016).
- [29] Stephanie Lin, Jacob Hilton, and Owain Evans. 2021. TruthfulQA: Measuring How Models Mimic Human Falsehoods. *arXiv preprint arXiv:2109.07958* (2021).
- [30] [n. d.]. Google cluster dataset. <https://github.com/google/cluster-data>
- [31] [n. d.]. Enengy dataset. <https://aemo.com.au/initiatives/major-programs/nem-distributed-energy-resources-der-program/>
- [32] [n. d.]. Edge infrastructure dataset. <https://aemo.com.au/energy-systems/energy/national-energy-market-nem/data-nem/data-dashboards-nem>.
- [33] [n. d.]. Data traffic of 5G communication. <https://standards.globalspec.com/standards/detail?docId=14565764>
- [34] Wen Chen, Jing Qiu, Junhua Zhao, Qingmian Chai, and Zhao Yang Dong. 2021. Bargaining Game-Based Profit Allocation of Virtual Power Plant in Frequency Regulation Market Considering Battery Cycle Life. *IEEE Transactions on Smart Grid* 12, 4 (2021), 2913–2928.
- [35] Shinji Mizuno and Florian Jarre. 1999. Global and polynomial-time convergence of an infeasible-interior-point algorithm using inexact computation. *Mathematical Programming* 84, 1 (1999), 105–122.
- [36] [n. d.]. Observability for NVIDIA NIM for LLMs. <https://docs.nvidia.com/nim/large-language-models/1.12.0/observability.html>
- [37] [n. d.]. LLM Energy. [Online]. Available: <https://www.globenewswire.com/news-release/2026/01/20/3221929/0/en/ESG-AI-Launches-Public-Calculator-Measuring-the-Environmental-Impact-of-Major-AI-Models.html>.
- [38] [n. d.]. Configuration of cloud server. <https://technical.city/zh/video/A800-PCIe-80-GB>
- [39] [n. d.]. Configuration of edge server. <https://www.hardware-corner.net/rtx-4090-llm-benchmarks/>
- [40] Zhizhen Li, Xuanhao Luo, Mingzhe Chen, Chenhan Xu, Shiwen Mao, and Yuchen Liu. 2025. Contextual Combinatorial Beam Management via Online Probing for Multiple Access mmWave Wireless Networks. *IEEE Journal on Selected Areas in Communications* 43, 3 (2025), 959–972.
- [41] Jia Liu, Jianbo Shao, Min Sheng, Yang Xu, Tarik Taleb, and Norio Shiratori. 2024. Mobile Crowdsensing Ecosystem With Combinatorial Multi-Armed Bandit-Based Dynamic Truth Discovery. *IEEE Transactions on Mobile Computing* 23, 12 (2024), 13095–13113.
- [42] Juncheng Wang, Yituo Liu, Ben Liang, and Min Dong. 2025. Constrained Over-the-Air Model Updating for Wireless Online Federated Learning with Delayed

7 结论与未来工作

分布式投机解码可以显著加快云 - 边缘环境中大型语言模型（LLM）的推理速度，但对其延迟、流量和能耗管理的全面研究，尤其是在电池供电的移动边缘计算环境中，仍然基本缺失。本文通过新颖的分析建模、算法设计、理论分析和实际实验填补了这一空白，重点在长期内以在线方式创新性地整合赌博学习和在线学习以优化系统。

未来的工作可能包括多个研究方向。我们打算为基于预测解码的云 - 边协作大语言模型推理设计激励机制和安全协议，并研究跨云与边界的其他类型大语言模型推理加速技术的优化。

致谢

本研究部分得到了西藏自治区科技项目（XZ202401ZY0027）、北京未来区块链与隐私计算前沿创新中心、北京市自然科学基金（2026FTZD001 和 4222033）、北京市多模态数据智能感知与治理重点实验室以及美国国家科学基金会（CNS-2047719 和 CNS-2225949）的支持。

参考文献

- [1] Zhi Yao、Zhiqing Tang、Wenmian Yang 和 Weijia Jia. 2025。通过云 - 边协作提升大语言模型服务质量：一种基于扩散的多智能体强化学习方法。IEEE 服务计算汇刊 18 卷，第 3 期（2025），1412–1427。[2] Yandi Li、Jianxiong Guo、Zhiqing Tang、Xingjian Ding、Juncheng Wang、Tian Wang 和 Weijia Jia. 2025。用于调度不可预测大语言模型请求的云 - 边系统：基于组合赌博机的方法。IEEE 服务计算汇刊 18 卷，第 6 期（2025），3567–3580。[3] Yaniv Leviathan、Matan Kalman 和 Yossi Matias. 2023。通过推测解码实现从变压器模型的快速推理。在 *ICML*。
- [4] Ying He、Jingcheng Fang、F. Richard Yu 和 Victor C. Leung。2024。云 - 边计算中大语言模型（LLMs）的推理卸载与资源分配：一种主动推理方法。IEEE 移动计算汇刊 23 卷，第 12 期（2024），11253–11264。
- [5] JIAHONG NING、Ce ZHENG 和 Tingting Y

- [14] 黄海洋，孟天辉，贾伟佳。2025. 边缘 - 云大模型系统中提示安全性与系统性能联合优化。载于 IEEE INFOCOM.[15] 徐大亮，尹望松，张浩，晋鑫，张颖，魏世云，徐梦威，刘轩泽。2025. EdgeLLM：基于设备的快速大模型推理与推测解码。IEEE 移动计算汇刊 24 卷，4 期 (2025)，3256–3273.[16] 叶胜远，欧阳贝，曾烈康，钱天一，朱晓文，唐健，陈旭。2025. Jupiter：边缘设备上生成型大模型的快速高效协作推理。载于 IEEE INFOCOM.[17] Ruslan Svirschevski, Avner May, 陈卓明，陈北迪，贾志浩，Max Ryabinin. 2024. SpecExec：面向消费设备的交互式大模型推理的大规模并行推测解码。载于 NeurIPS.[18] 邓秀梅，李俊，石龙，魏志强，周晓波，袁金洪。2022. 无线供能移动边缘计算：动态资源分配

- Information. In *IEEE INFOCOM*.
- [43] Rui Li, Tao Ouyang, Liekang Zeng, Guocheng Liao, Zhi Zhou, and Xu Chen. 2024. Online Optimization of DNN Inference Network Utility in Collaborative Edge Computing. *IEEE/ACM Transactions on Networking* 32, 5 (2024), 4414–4426.
- [44] [n. d.]. Gurobi. <http://www.gurobi.com>
- [45] Xuanyu Cao, Junshan Zhang, and H. Vincent Poor. 2018. A Virtual-Queue-Based Algorithm for Constrained Online Convex Optimization With Applications to Data Center Resource Allocation. *IEEE Journal of Selected Topics in Signal Processing* 12, 4 (2018), 703–716.

Appendix

A. Proof of Theorem 4.1

Let $\omega_{nt} = \sum_{j \in \mathcal{F}_{nt}} v_{nt}^j$ and $q_{nt}^j = v_{nt}^j / \omega_{nt}$, then we have

$$\frac{\omega_{n,t+1}}{\omega_{nt}} \leq 1 - \frac{\varsigma}{\gamma_{max}I} \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j + \frac{\gamma^2}{2\gamma_{max}^2 I^2} \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^{j^2},$$

summing over T and taking ln, then it leads to

$$\ln \omega_{n,T+1} \leq \ln \gamma_{max} + \sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} \left(\frac{\varsigma^2}{2\gamma_{max}^2 I^2} q_{nt}^j \ell_{nt}^{j^2} - \frac{\varsigma}{\gamma_{max}I} q_{nt}^j \ell_{nt}^j \right).$$

As $v_{j^*,T+1} = \exp(-\frac{\varsigma}{\gamma_{max}I} \sum_{t=1}^T \ell_{nt}^{j^*}) \leq \omega_{n,T+1}$, where j^* is the optimum, we can get $-\frac{\varsigma}{\gamma_{max}I} \sum_{t=1}^T \ell_{nt}^{j^*} \leq \ln \omega_{n,T+1}$. Then we can obtain

$$\sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j - \sum_{t=1}^T \ell_{nt}^{j^*} \leq \frac{\gamma_{max}I \ln \gamma_{max}}{\varsigma} + \frac{\varsigma}{2\gamma_{max}I} \sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^{j^2}.$$

Using $\mathbb{E}[\ell_{nt}^{j^2}] \leq I^2/p_{nt}^j$ and $p_{nt}^j \geq (1-\varsigma)q_{nt}^j$, we acquire

$$\mathbb{E}[\sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j] - \sum_{t=1}^T \ell_{nt}^{j^*} \leq \frac{\gamma_{max}I \ln \gamma_{max}}{\varsigma} + \frac{\varsigma TI}{2(1-\varsigma)}.$$

Since $p_{nt}^j = (1-\varsigma)q_{nt}^j + \varsigma/\gamma_{max}$, we can deduce that $\sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j = \frac{\mathbb{E}[\ell_{nt}^j] - \frac{\varsigma}{\gamma_{max}} \sum_{j \in \mathcal{F}_{nt}} \ell_{nt}^j}{1-\varsigma}$. Due to $\bar{y}_n \leftarrow \bar{j}$, $\gamma_n^* \leftarrow j^*$, and using the inequality $\mathbb{E}[\sum_{t=1}^T \frac{1}{\gamma_{max}} \sum_{j \in \mathcal{F}_{nt}} \ell_{nt}^j] \leq TI$, we get the following:

$$\mathbb{E}[\mathcal{P}_1(\bar{y}_{nt})] - \mathcal{P}_1(\gamma_n^*) \leq \gamma_{max}I \ln \gamma_{max} / \varsigma + 3\varsigma TI / 2,$$

considering $\varsigma = \sqrt{2\gamma_{max} \ln \gamma_{max} / 3T}$ and $n \in \mathcal{N}$, we get $\mathcal{R}_1$.

B. Proof of Theorem 4.2

To facilitate our proof, we first make several technical *assumptions* which are generally easy to be satisfied for all $t \leq T$: (i) There exists some $\phi > 0$ such that each g_t is Lipschitz continuous with modulus ϕ , i.e., $\|g_t(\omega_1) - g_t(\omega_2)\|_2 \leq \phi \|\omega_1 - \omega_2\|_2$, for any $\omega_1, \omega_2 \in \mathbb{W}$. (ii) There exists constant $\delta_1 > 0$ such that $\|\omega\|_2 \leq \delta_1$ for any $\omega \in \mathbb{W}$. (iii) There exists constant $\delta_2 > 0$ such that $|f_t(\omega)| \leq \delta_2$ for any $\omega \in \mathbb{W}$. (iv) There exists constant $\delta_3 > 0$ such that $\|g_t(\omega)\|_\infty \leq \delta_3$ for each $\omega \in \mathbb{W}$. (v) The algorithmic parameter μ and Lipschitz modulus ϕ satisfy $\mu \geq \phi^2$. Then we define some *notations* to quantify the violation from different perspectives for $\forall t$: (i) $v_\omega(t) \triangleq \sum_{\tau=0}^{t-1} \|\omega_{\tau+1}^* - \omega_\tau^*\|_2$. (ii) $v_f(t) \triangleq \sum_{\tau=0}^{t-1} \|f_\tau - f_{\tau+1}\|_\infty$. (iii) $v_g(t) \triangleq \sum_{\tau=1}^{t-1} \|g_{\tau-1} - g_\tau\|_\infty$. (iv) $\bar{v}_g(t) \triangleq \sum_{\tau=1}^{t-1} \|g_{\tau-1} - g_\tau\|_\infty$. (v) $v_\lambda(t) \triangleq \sum_{\tau=0}^{t-2} \|\lambda_\tau^* - \lambda_{\tau+1}^*\|_2$. Here, the optimal primal and dual variable are denoted as ω_τ^* and λ_τ^* , respectively.

According to equation (7), we have $\lambda_{n,t+1} \geq 0$. Then, for $\forall t \leq T$, $\lambda_{n,t+1} + g_{nt}(\omega_{t+1}) \geq 0$. If $g_{nt}(\omega_{t+1}) \geq 0$, we will gain

$$\lambda_{n,t+1} \geq \lambda_{nt} + g_{nt}(\omega_{t+1}) \geq g_{nt}(\omega_{t+1}) = |g_{nt}(\omega_{t+1})|,$$

If $g_{nt}(\omega_{t+1}) < 0$, then we can acquire

$$\lambda_{n,t+1} \geq -g_{nt}(\omega_{t+1}) = |g_{nt}(\omega_{t+1})|,$$

which leads to $\lambda_{n,t+1}^2 \geq g_{nt}^2(\omega_{t+1})$. Summing this inequation over all n yields $\|\lambda_{t+1}\|_2 \geq \|g_t(\omega_{t+1})\|_2$. Then, in view of the work [45], for $\forall t$, we are able to get the following inequality:

$$\sum_{\tau=0}^{t-1} f_\tau(\omega_\tau) \leq \sum_{\tau=0}^{t-1} f_\tau(\omega_\tau^*) + \frac{1}{2} \|g_{t-1}(\omega_t)\|_2^2 - \frac{1}{2} \|\lambda_t\|_2^2 + v_f(t) \\ + v_g(t + 4\mu\delta_1 v_\omega(t) + \|g_0(\omega_0)\|_2^2 + \delta_2 + f_0(\omega_0) + \mu \|\omega_0 - \omega_0^*\|_2^2).$$

Substituting $\|\lambda_t\|_2 \geq \|g_{t-1}(\omega_t)\|_2$ into this inequality gives

$$\mathcal{R}_2 \leq v_f(T) + v_g(T) + 4\mu\delta_1 v_\omega(T) + \delta_2 \\ + \|g_0(\omega_0)\|_2^2 + f_0(\omega_0) + \mu \|\omega_0 - \omega_0^*\|_2^2.$$

Due to v_f, v_g, v_ω are sub-linear in $t \leq T$, then $\mathcal{R}_2 \leq o(T)$.

C. Proof of Theorem 4.3

Based on equation (7), we have $\lambda_{nt} \geq \sum_{\tau=0}^{t-1} g_{n\tau}(\omega_{\tau+1})$, $\forall t \leq T$. Therefore, we can obtain the following inequation, $\forall t \leq T$:

$$\sum_{\tau=0}^{t-1} g_{n\tau}(\omega_\tau) \leq \sum_{\tau=0}^{t-2} |g_{\tau+1,k}(\omega_{\tau+1}) - g_{n\tau}(\omega_{\tau+1})| \\ + g_{n0}(\omega_0) + \sum_{\tau=0}^{t-2} g_{n\tau}(\omega_{\tau+1}) \leq g_{n0}(\omega_0) + \lambda_{n,t-1} + \bar{v}_g(t)$$

As $\lambda_{n,t-1} \leq \|\lambda_{t-1}\|_2$, then, for $\forall t \leq T$, we can gain

$$\sum_{\tau=0}^{t-1} g_{n\tau}(\omega_\tau) \leq g_{n0}(\omega_0) + \bar{v}_g(t) + \|\lambda_{t-1}\|_2.$$

Based on the previous work [45], for $\forall t \leq T$, we will have

$$\|\lambda_t\|_2 \leq 4v_\lambda(t) + 2\sqrt{v_f(t) + 2v_g(t) + 8\mu\delta_1 v_x(t)} \\ + \sqrt{\delta_3^2 + 4\delta_2 + 2\mu \|\omega_0 - \omega_0^*\|_2^2 + 2\|g_0(\omega_0)\|_2^2 + 2\|\lambda_0^*\|_2}.$$

Making use of this inequation and noting that the R.H.S. for it is monotonically increasing with $t \leq T$, we get the bound:

$$\mathcal{V} \leq \bar{v}_g(T) + 4v_\lambda(T) + 2\sqrt{v_f(T) + 2v_g(T)} \\ + \sqrt{\delta_3^2 + 4\delta_2 + 2\mu \|\omega_0 - \omega_0^*\|_2^2 + 2\|g_0(\omega_0)\|_2^2} \\ + 2\|\lambda_0^*\|_2 + g_{n0}(\omega_0) + \sqrt{8\mu\delta_1 v_\omega(T)}$$

Considering that $\bar{v}_g$ and v_λ are sub-linear in $t \leq T$ while v_f, v_g and v_ω are subquadratic in $t \leq T$, we have $\mathcal{V} \leq o(T)$.

D. Proof of Theorem 4.4

We use $\bar{y}_{nt}, \bar{\omega}_t$ to denote the online decisions obtained from our algorithms, and $\hat{y}_n, \hat{\omega}$ to denote the static offline optimal decisions for the original problem $\mathcal{P}$ over the whole time horizon. We have

$$\mathcal{R} = \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathcal{P}(\hat{y}_n, \hat{\omega}) = \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] \\ + \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] - \mathcal{P}(\gamma_n^*, \hat{\omega}) + \mathcal{P}(\gamma_n^*, \hat{\omega}) - \mathcal{P}(\hat{y}_n, \hat{\omega}).$$

As the expectation is only for $\bar{y}_{nt}$, the first two terms are bounded by Theorem 4.2, where ω_t^* is the optimum at t given $\bar{y}_{nt}$:

$$\mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] \leq \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \omega_t^*)] \leq o(T).$$

The next two terms are bounded by Theorem 4.1:

$$\mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] - \mathcal{P}(\gamma_n^*, \hat{\omega}) \leq o(T).$$

The last two terms can also be bounded:

$$\mathcal{P}(\gamma_n^*, \hat{\omega}) - \mathcal{P}(\hat{y}_n, \hat{\omega}) = \mathcal{P}_1(\gamma_n^*) - \mathcal{P}_1(\hat{y}_n) \leq 0,$$

because γ_n^* statically minimizes $\mathcal{P}_1$. That is, overall, we obtain

$$\mathcal{R} = \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathcal{P}(\hat{y}_n, \hat{\omega}) \leq o(T) + o(T) + 0 = o(T).$$

信息。发表于 *IEEE INFOCOM*. [43] Rui Li, Tao Ouyang, Liekang Zeng, Guocheng Liao, Zhi Zhou, and Xu Chen. 2024. 协作边缘计算中 DNN 推理网络效用的在线优化. *IEEE/ACM 网络学报* 32 卷, 第 5 期 (2024), 4414–4426. [44] [无日期]. Gurobi. <http://www.gurobi.com> [45] Xuanyu Cao, Junshan Zhang, 和 H. Vincent Poor. 2018. 一种基于虚拟队列的受约束在线凸优化算法及其在数据中心资源分配中的应用. *IEEE 信号处理选題期刊* 12 卷, 第 4 期 (2018), 703–716.

附录

A. 定理 4.1 的证明

让 $\omega_{nt} = \sum_{j \in \mathcal{F}_{nt}} v_{nt}^j$ 和 $q_{nt}^j = v_{nt}^j / \omega_{nt}$, 那么我们有

$$\frac{\omega_{n,t+1}}{\omega_{nt}} \leq 1 - \frac{\varsigma}{\gamma_{max}I} \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j + \frac{\gamma^2}{2\gamma_{max}^2 I^2} \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^{j^2},$$

summing over T and taking ln, then it leads to

$$\ln \omega_{n,T+1} \leq \ln \gamma_{max} + \sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} \left(\frac{\varsigma^2}{2\gamma_{max}^2 I^2} q_{nt}^j \ell_{nt}^{j^2} - \frac{\varsigma}{\gamma_{max}I} q_{nt}^j \ell_{nt}^j \right).$$

作为 $v_{j^*,T+1} = \exp(-\frac{\varsigma}{\gamma_{max}I} \sum_{t=1}^T \ell_{nt}^{j^*}) \leq \omega_{n,T+1}$, 其中 j^* 是最优值, 我们可以得到 $-\frac{\varsigma}{\gamma_{max}I} \sum_{t=1}^T \ell_{nt}^{j^*} \leq \ln \omega_{n,T+1}$. 然后我们可以得到

$$\sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j - \sum_{t=1}^T \ell_{nt}^{j^*} \leq \frac{\gamma_{max}I \ln \gamma_{max}}{\varsigma} + \frac{\varsigma}{2\gamma_{max}I} \sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^{j^2}.$$

使用 $\mathbb{E}[\ell_{nt}^j] \leq I^2/p_{nt}^j$ 和 $p_{nt}^j \geq (1-\varsigma)q_{nt}^j$, 我们获得

$$\mathbb{E}[\sum_{t=1}^T \sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j] - \sum_{t=1}^T \ell_{nt}^{j^*} \leq \frac{\gamma_{max}I \ln \gamma_{max}}{\varsigma} + \frac{\varsigma TI}{2(1-\varsigma)}.$$

自 $p_{nt}^j = (1-\varsigma)q_{nt}^j + \varsigma/\gamma_{max}$ 起, 我们可以推断 $\sum_{j \in \mathcal{F}_{nt}} q_{nt}^j \ell_{nt}^j = \frac{\mathbb{E}[\ell_{nt}^j] - \frac{\varsigma}{\gamma_{max}} \sum_{j \in \mathcal{F}_{nt}} \ell_{nt}^j}{1-\varsigma}$. 由于 $\bar{y}_n \leftarrow \bar{j}$, $\gamma_n^* \leftarrow j^*$, 并使用不等式 $\mathbb{E}[\sum_{t=1}^T \frac{1}{\gamma_{max}} \sum_{j \in \mathcal{F}_{nt}} \ell_{nt}^j] \leq TI$, 我们得到如下结果:

$$\mathbb{E}[\mathcal{P}_1(\bar{y}_{nt})] - \mathcal{P}_1(\gamma_n^*) \leq \gamma_{max}I \ln \gamma_{max} / \varsigma + 3\varsigma TI / 2,$$

考虑 $\varsigma = \sqrt{2\gamma_{max} \ln \gamma_{max} / 3T}$ 和 $n \in \mathcal{N}$, 我们得到 $\mathcal{R}_1$.

B. 定理 4.2 的证明

为了便于我们的证明, 我们首先做出几个技术上的假设, 这些假设通常对于所有 $t \leq T$ 都容易满足: (i) 存在某个 $\phi > 0$ 使得每个 g_t 都是模量为 ϕ 的 Lipschitz 连续函数, 即 $\|g_t(\omega_1) - g_t(\omega_2)\|_2 \leq \phi \|\omega_1 - \omega_2\|_2$, 对于任意 $\omega_1, \omega_2 \in \mathbb{W}$. (ii) 存在常数 $\delta_1 > 0$ 使得 $\|\omega\|_2 \leq \delta_1$ 对于任意 $\omega \in \mathbb{W}$. (iii) 存在常数 $\delta_2 > 0$ 使得 $|f_t(\omega)| \leq \delta_2$ 对于任意 $\omega \in \mathbb{W}$. (iv) 存在常数 $\delta_3 > 0$ 使得 $\|g_t(\omega)\|_\infty \leq \delta_3$ 对于每个 $\omega \in \mathbb{W}$. (v) 算法参数 μ 和 Lipschitz 模量 ϕ 满足 $\mu \geq \phi^2$. 然后我们定义一些符号来从不同角度量化 $\forall t$ 的偏差: (i) $v_\omega(t) \triangleq \sum_{\tau=0}^{t-1} \|\omega_{\tau+1}^* - \omega_\tau^*\|_2$. (ii) $v_f(t) \triangleq \sum_{\tau=0}^{t-1} \|f_\tau - f_{\tau+1}\|_\infty$. (iii) $v_g(t) \triangleq \sum_{\tau=1}^{t-1} \|g_{\tau-1} - g_\tau\|_\infty$. (iv) $\bar{v}_g(t) \triangleq \sum_{\tau=1}^{t-1} \|g_{\tau-1} - g_\tau\|_\infty$. (v) $v_\lambda(t) \triangleq \sum_{\tau=0}^{t-2} \|\lambda_\tau^* - \lambda_{\tau+1}^*\|_2$. 这里, 最优的原始变量和对偶变量分别表示为 ω_τ^* 和 λ_τ^* .

根据方程式 (7), 我们有 $\lambda_{n,t+1} \geq 0$. 然后, 对于 $\forall t \leq T, \lambda_{n,t+1} + g_{nt}(\omega_{t+1}) \geq 0$. 如果 $g_{nt}(\omega_{t+1}) \geq 0$, 我们将获得

$$\lambda_{n,t+1} \geq \lambda_{nt} + g_{nt}(\omega_{t+1}) \geq g_{nt}(\omega_{t+1}) = |g_{nt}(\omega_{t+1})|,$$

如果 $g_{nt}(\omega_{t+1}) < 0$, 那么我们可以获得

$$\lambda_{n,t+1} \geq -g_{nt}(\omega_{t+1}) = |g_{nt}(\omega_{t+1})|,$$

这导致了 $\lambda_{n,t+1}^2 \geq g_{nt}^2(\omega_{t+1})$. 将这个不等式对所有 n 求和得到

$\|\lambda_{t+1}\|_2 \geq \|g_t(\omega_{t+1})\|_2$. 然后, 鉴于 [45], 对 $\forall t$ 的工作, 我们能够得到以下不等式:

$$\sum_{\tau=0}^{t-1} f_\tau(\omega_\tau) \leq \sum_{\tau=0}^{t-1} f_\tau(\omega_\tau^*) + \frac{1}{2} \|g_{t-1}(\omega_t)\|_2^2 - \frac{1}{2} \|\lambda_t\|_2^2 + v_f(t) \\ + v_g(t + 4\mu\delta_1 v_\omega(t) + \|g_0(\omega_0)\|_2^2 + \delta_2 + f_0(\omega_0) + \mu \|\omega_0 - \omega_0^*\|_2^2).$$

Substituting $\|\lambda_t\|_2 \geq \|g_{t-1}(\omega_t)\|_2$ into this inequality gives

$$\mathcal{R}_2 \leq v_f(T) + v_g(T) + 4\mu\delta_1 v_\omega(T) + \delta_2 \\ + \|g_0(\omega_0)\|_2^2 + f_0(\omega_0) + \mu \|\omega_0 - \omega_0^*\|_2^2.$$

由于 v_f, v_g, v_ω are sub-linear in $t \leq T$, then $\mathcal{R}_2 \leq o(T)$.

C. 定理 4.3 的证明

根据方程 (7), 我们有 $\lambda_{nt} \geq \sum_{\tau=0}^{t-1} g_{n\tau}(\omega_{\tau+1})$, $\forall t \leq T$. 因此, 我们可以得到以下不等式, $\forall t \leq T$:

$$\sum_{\tau=0}^{t-1} g_{n\tau}(\omega_\tau) \leq \sum_{\tau=0}^{t-2} |g_{\tau+1,k}(\omega_{\tau+1}) - g_{n\tau}(\omega_{\tau+1})| \\ + g_{n0}(\omega_0) + \sum_{\tau=0}^{t-2} g_{n\tau}(\omega_{\tau+1}) \leq g_{n0}(\omega_0) + \lambda_{n,t-1} + \bar{v}_g(t)$$

作为 $\lambda_{n,t-1} \leq \|\lambda_{t-1}\|_2$, then, for $\forall t \leq T$, we can gain

$$\sum_{\tau=0}^{t-1} g_{n\tau}(\omega_\tau) \leq g_{n0}(\omega_0) + \bar{v}_g(t) + \|\lambda_{t-1}\|_2.$$

基于之前为 $\forall t \leq T$ 所做的 [45], 工作, 我们将有

$$\|\lambda_t\|_2 \leq 4v_\lambda(t) + 2\sqrt{v_f(t) + 2v_g(t) + 8\mu\delta_1 v_x(t)} \\ + \sqrt{\delta_3^2 + 4\delta_2 + 2\mu \|\omega_0 - \omega_0^*\|_2^2 + 2\|g_0(\omega_0)\|_2^2 + 2\|\lambda_0^*\|_2}.$$

Making use of this inequation and noting that the R.H.S. for it is monotonically increasing with $t \leq T$, we get the bound:

$$\mathcal{V} \leq \bar{v}_g(T) + 4v_\lambda(T) + 2\sqrt{v_f(T) + 2v_g(T)} \\ + \sqrt{\delta_3^2 + 4\delta_2 + 2\mu \|\omega_0 - \omega_0^*\|_2^2 + 2\|g_0(\omega_0)\|_2^2} \\ + 2\|\lambda_0^*\|_2 + g_{n0}(\omega_0) + \sqrt{8\mu\delta_1 v_\omega(T)}$$

考虑到 $\bar{v}_g$ 和 v_λ 在 $t \leq T$ 中是次线性的, 而 v_f, v_g 和 v_ω 在 $t \leq T$ 中是次二次的, 我们有 $\mathcal{V} \leq o(T)$.

D. 定理 4.4 的证明

我们使用 $\bar{y}_{nt}, \bar{\omega}_t$ 来表示从我们的算法中获得的在线决策, 使用 $\hat{y}_n, \hat{\omega}$ 来表示原始问题 $\mathcal{P}$ 在整个时间范围内的静态离线最优决策。我们有

$$\mathcal{R} = \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathcal{P}(\hat{y}_n, \hat{\omega}) = \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] \\ + \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] - \mathcal{P}(\gamma_n^*, \hat{\omega}) + \mathcal{P}(\gamma_n^*, \hat{\omega}) - \mathcal{P}(\hat{y}_n, \hat{\omega}).$$

As the expectation is only for $\bar{y}_{nt}$, the first two terms are bounded by Theorem 4.2, where ω_t^* is the optimum at t given $\bar{y}_{nt}$:

$$\mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] \leq \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \omega_t^*)] \leq o(T).$$

接下来的两个项由定理 4.1 给出界限:

$$\mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \hat{\omega})] - \mathcal{P}(\gamma_n^*, \hat{\omega}) \leq o(T).$$

最后两个项也可以被界定:

$$\mathcal{P}(\gamma_n^*, \hat{\omega}) - \mathcal{P}(\hat{y}_n, \hat{\omega}) = \mathcal{P}_1(\gamma_n^*) - \mathcal{P}_1(\hat{y}_n) \leq 0,$$

因为 γ_n 静态地最小化 $\mathcal{P}_1$ 。也就是说, 总的来说, 我们得到

$$\mathcal{R} = \mathbb{E}[\mathcal{P}(\bar{y}_{nt}, \bar{\omega}_t)] - \mathcal{P}(\hat{y}_n, \hat{\omega}) \leq o(T) + o(T) + 0 = o(T).$$